

# 基于动态规划算法的 NIPT 检测策略与女胎异常识别方法研究

## 摘要

本文研究男胎孕妇无创产前检测 (NIPT) 最佳时点选择与女胎异常判定问题, 建立了多因素驱动的风险评估等模型, 采用了改进动态规划算法求解, 计算结果对提升临床检测准确性和降低治疗风险具有重要意义。

**针对问题一**, 本文选取孕妇年龄、孕妇 BMI、检测孕周、13 号染色体的 Z 值、18 号染色体的 Z 值以及 21 号染色体的 Z 值作为男胎 Y 染色体浓度的 **6 个核心影响因素**, 明确变量间呈非线性关系, 建立了**梯度提升回归树模型**。之后引入 5 折交叉验证, 以  $RMSE = 0.031$ 、 $R^2_{OOF} = 0.136$  验证了模型拟合与泛化性能良好。最后再采用基于 Monte Carlo 的置换策略对特征重要性进一步挖掘, 最后得到**孕妇 BMI 与检测孕周影响最大**。

**针对问题二**, 本文构建了一个**风险函数为目标的优化模型**, 综合考虑“早检失败重测成本”与“晚检治疗风险”, 并引入 BMI 越大, 普遍达标孕周越晚的规律。在此基础上为了控制首检失败率, 本文人为增加设定了组内首检时点达标率  $\geq 0.95$  的临床安全约束条件。最后本文使用**动态规划算法**求解得到最优分组切点及各组最优首检时点, 全体平均风险达 **0.3604**, 同时进行收敛性与敏感性分析验证方案稳健。

**针对问题三**, 本文延用问题二的优化模型, 并在此基础上纳入孕妇年龄、染色体 Z 值及检测误差等多因素, 建立了扩展的协变量时间事件模型和误差修正达标率函数。之后本文采用**前缀和优化的动态规划算法**进行高效求解, 最终将样本划分为 **5 个 BMI 区间**并确定了各组统一推荐检测孕周, 同时给出了**高龄孕妇组内检测后移**的调整建议。敏感性分析表明该多因素优化方案在不同误差扰动下均保持稳健。

**针对问题四**, 由于女胎没有具体的指标去确保 NIPT 的结果是否基本准确, 本文首先利用男胎阴性样本建立正常参照体系, 进而融合测序质量指标与多染色体 Z 值等多源证据, 并通过**概率校准构建异常评分模型**; 在此基础上, 采用**分层阈值策略**以控制假阳性率, 确保判定结果的稳健性与可迁移性。最终, 在 605 例女胎样本中有效判定出**异常 84 例**, 结果表明该方法在提升女胎染色体异常检测准确性与可解释性方面具有显著效果。

**本文的亮点在于**: 采用梯度提升回归树捕捉变量间非线性关系, 结合动态规划与前缀和优化实现高效率求解多因素风险最小化分组及各组最优首检时点, 并通过分层阈值与概率校准提升女胎异常判定的稳健性与可迁移性。

**关键字**: NIPT 梯度提升回归树 改进动态规划 概率校准

## 一、问题重述

### 1.1 问题背景

无创产前检测（NIPT）通过采集母体血液检测胎儿游离 DNA 片段，可早期筛查胎儿 21 号、18 号、13 号染色体异常。但其准确性依赖胎儿性染色体浓度，且检测仅能在孕期 10-25 周开展，其中胎儿异常发现时机直接关联治疗风险。同时，男胎 Y 染色体浓度与孕妇孕周、BMI 紧密相关。然而，传统经验性 BMI 分组及统一检测时点，无法适配孕妇年龄、身高、孕情等个体差异，易导致测序失败与重复检测现象；此外，女胎因无 Y 染色体，其染色体异常判定缺乏系统方法。因此，对影响检测的因素、最佳检测时点和胎儿异常判定进行深入、准确的分析，对于减低治疗窗口期缩短风险和临床精准检测与决策至关重要。

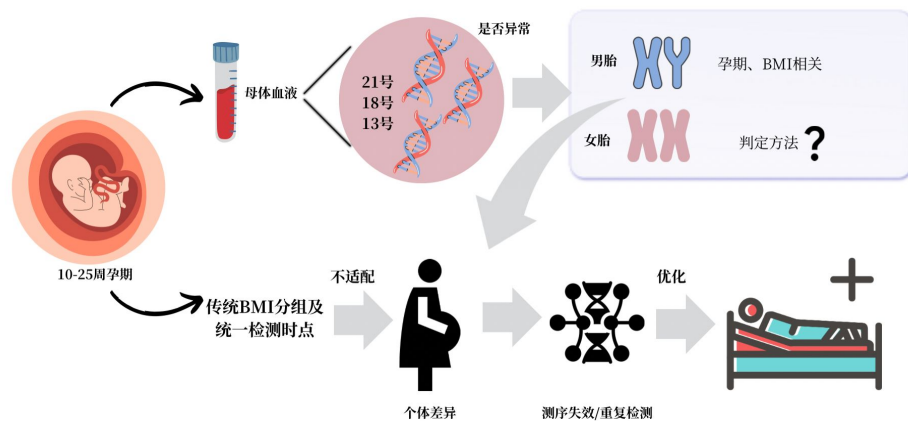


图 1 NIPT 无创产前检测示意图

### 1.2 问题提出

基于上述背景，需利用附件数据建立数学模型解决以下问题：

**问题一：**分析胎儿 Y 染色体浓度与孕妇孕周数、BMI 等指标的相关特性，构建关系模型并检验其显著性。

**问题二：**以男胎孕妇 BMI 为主要影响因素，对其进行合理分组，确定每组 BMI 区间与最佳 NIPT 时点以最小化潜在风险，并分析检测误差的影响。

**问题三：**综合孕妇身高、体重、年龄等因素、检测误差及 Y 染色体浓度达标比例，结合 BMI 对男胎孕妇合理分组，确定每组最佳 NIPT 时点以最小化风险，并分析检测误差的影响。

**问题四:** 以女胎孕妇 21 号、18 号、13 号染色体非整倍体为判定结果, 综合 X 染色体及上述染色体的 Z 值、GC 含量、读段数及相关比例、BMI 等因素, 给出女胎异常的判定方法。

针对以上问题, 本文采用了以下总体思路框架进行系统性研究:

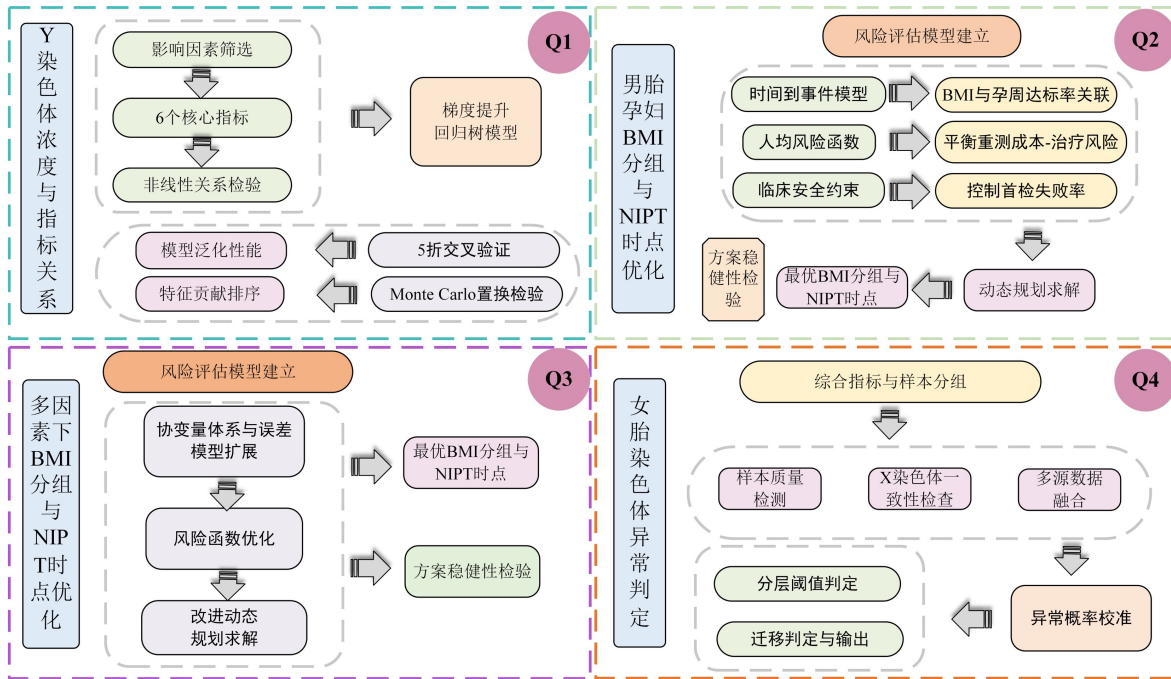


图 2 总体思路框架图

## 二、模型假设

为简化问题, 本文做出以下假设:

- 本文假设前三个问题中, 男胎 GC 含量不存在过高、过低或分布异常, 所有样本可以认定测序质量不存在问题。
- 本文假设达标时间  $T^*$  的分布满足  $F(t|\cdot)$  对孕周  $t$  单调不减, 且存在合理右界  $H$ 。
- 本文假设最后一问女胎的正常样本在融合特征空间与男胎阴性近似同分布, 以男胎学习得到的概率标尺与分层阈值可迁移用于女胎判定。
- 本文假设附件数据是可信的, 不存在人为实验操作有误带来的影响。

### 三、符号说明

| 符号                       | 说明                | 单位                |
|--------------------------|-------------------|-------------------|
| $n$                      | 样本量               | 例                 |
| $k$                      | BMI 分组数           | —                 |
| $a$                      | 孕妇年龄              | 岁                 |
| $b$                      | 孕妇 BMI            | kg/m <sup>2</sup> |
| $z_{13}, z_{18}, z_{21}$ | 13/18/21 号染色体 Z 值 | —                 |
| $Z_X$                    | X 染色体 Z 值         | —                 |
| $y, \hat{y}$             | 胎儿 Y 浓度及其预测值      | 比例                |
| $T^*$                    | 首次达标孕周            | 周                 |

注：表中未说明的符号以首次出现处为准

### 四、问题一的模型的建立和求解

#### 4.1 问题分析

本问旨在研究胎儿 Y 染色体浓度与孕妇 BMI 等指标的相关特性。首先，结合文献理论与附件 GC 含量数据实际，本文确定了以孕妇年龄、孕妇 BMI、检测孕周、13 号染色体的 Z 值、18 号染色体的 Z 值以及 21 号染色体的 Z 值作为核心影响因素。

为进一步探讨胎儿 Y 染色体浓度与 6 个核心指标的关联类型，本文通过绘制数据之间的散点图进行检验，明确变量之间呈现非线性关系，构建了以 6 个相关指标作为自变量，以胎儿 Y 染色体浓度作为因变量的梯度提升回归树模型。之后本文使用 5 折交叉验证了模型拟合与泛化性能良好，并且再进一步结合 Monte-Carlo 置换检验与特征重要性评估，验证了模型显著性并得到了孕妇 BMI 与检测孕周贡献最大。

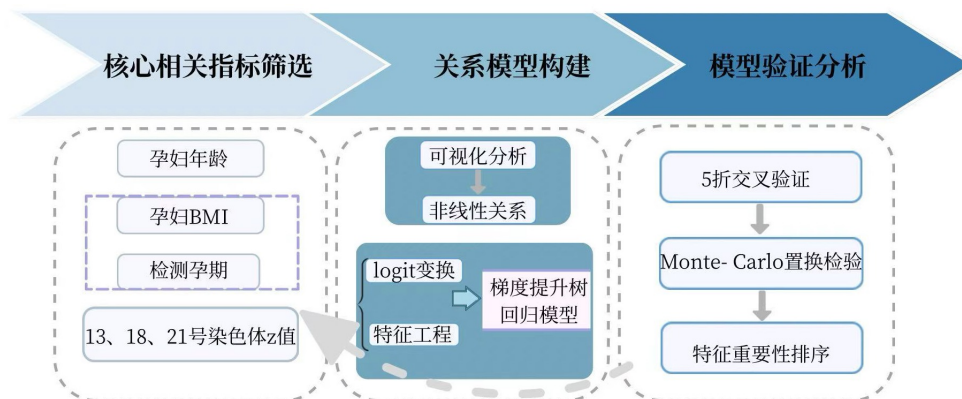


图3 问题一思路图

4.2 胎儿 Y 染色体浓度与相关指标关系模型的建立与求解

4.2.1 核心影响因素筛选

本文首先以“胎儿 Y 染色体浓度”为响应变量，对全部候选自变量开展斯皮尔曼秩相关检验，以刻画可能存在的非线性单调关系并提升对异常值的鲁棒性。结合相关系数的大小与显著性，并在孕周/BMI 分层下复核稳健性，筛除贡献弱或不稳定的指标。

在进入建模前对数据质量进行核查，发现 GC 含量整体稳定在合理区间（约 40%），且与响应的相关性弱、方向不一致，判定其更适合作为质控参照而非核心自变量。

综合上述检验，本文最终确定孕妇年龄、孕妇 BMI、检测孕周、以及 13/18/21 号染色体的 Z 值为影响胎儿 Y 染色体浓度的 6 个核心因素；这一结论与现有研究报道相符 [1][2]。

4.2.2 非线性关系检验

为了深入研究胎儿 Y 染色体浓度与 6 个核心指标关系，先对数据进行标准化处理，再根据提供的样本对应关系绘画出散点矩阵图如下所示：

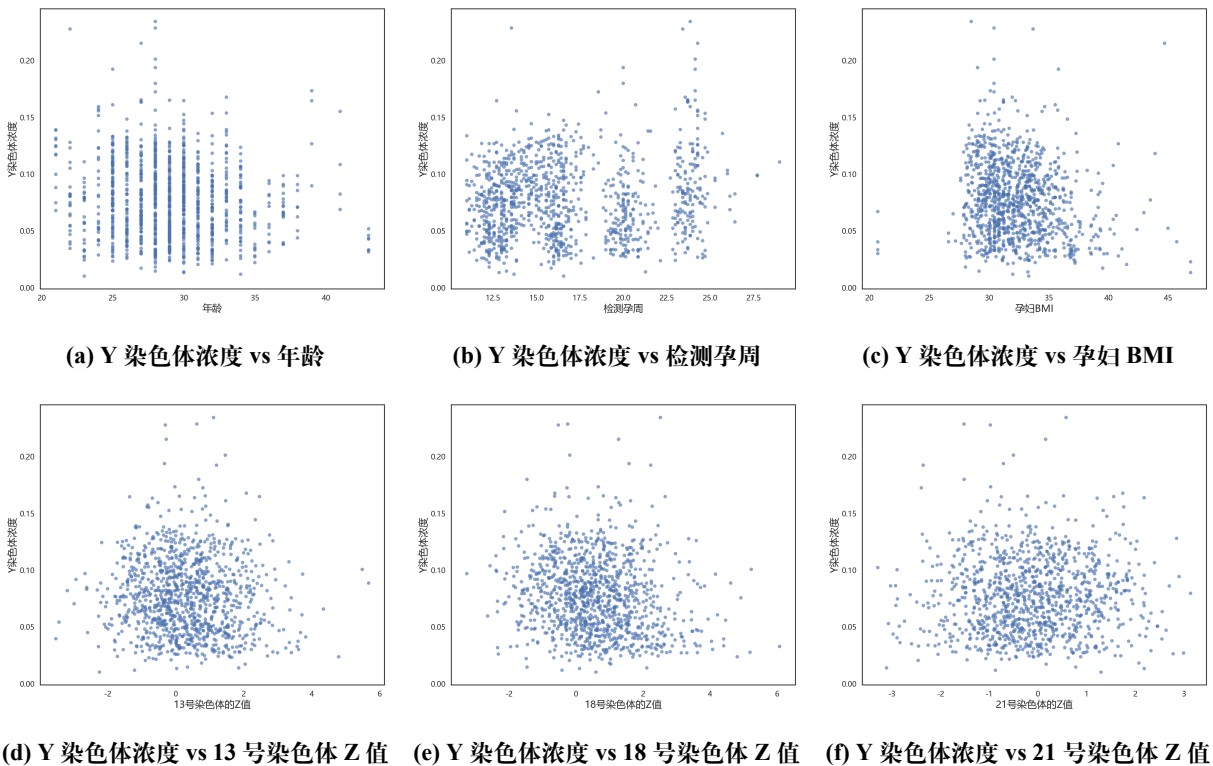


图 4 Y 染色体浓度与 6 个相关影响因素的散点图

结果显示，Y 染色体浓度与各指标间无线性趋势。例如，孕妇 BMI 与 Y 染色体浓度呈非单调变化、检测孕周与 Y 染色体浓度无直线关联。因此排除线性回归方法，本文确定采用监督式多元非线性回归的方法来建立关系模型。

### 4.2.3 数据预处理

假设共有  $n$  个样本，第  $i$  个样本的因变量为胎儿 Y 染色体浓度为变量  $y_i$ （取值 0-1）。自变量向量为：

$$x_i = [a_i, b_i, t_i, z_{13,i}, z_{18,i}, z_{21,i}]^T$$

#### (1) 目标数据变换

为了避免模型训练中输出越界、端点误差权重失衡等问题。在此先对  $y_i$  进行轻微裁剪，再通过 logit 变换，转换目标变量，训练后反变换还原，有效保障数值优化稳定性。具体实现公式如下：

$$\begin{cases} y_i^{\text{clip}} &= \min \{ \max(y_i, \varepsilon), 1 - \varepsilon \}, \quad \varepsilon = 10^{-6} \\ z_i &= \text{logit}(y_i^{\text{clip}}) = \ln \left( \frac{y_i^{\text{clip}}}{1 - y_i^{\text{clip}}} \right) \end{cases} \quad (1)$$

#### (2) 特征工程优化

针对年龄、BMI、孕周的轻度偏态及染色体 Z 值的长尾 / 离群值，易主导模型训练问题，先对每列特征  $x_{.j}$  计算经验分位  $p = \hat{F}_j(x)$ ，再映射到标准正态分位点：

$$x_{ij}^{(\text{RG})} = \Phi^{-1} \left( \hat{F}_j(x_{ij}) \right) \quad (2)$$

其中  $\Phi^{-1}$  为标准正态的分位函数。随后对数据进行标准化并做二次多项式扩展，成功捕捉潜在的非线性与低阶交互效应，显著提升模型拟合能力。

### 4.2.4 梯度提升回归树模型建立

结合本问 6 个核心变量、样本量中等的数特点，梯度提升回归树（GBDT）在小规模数据上表现稳定<sup>[3]</sup>，既优于强线性假设的多元线性回归，又无需深度网络的大样本支撑，且能通过集成学习降低过拟合风险，因此本文选择其构建非线性回归模型<sup>[4]</sup>，具体求解步骤如下：

step1：基线初始化

以训练集目标变量均值作为初始模型值作：

$$f_0(\tilde{x}) = \bar{z} = \frac{1}{n} \sum_{i=1}^n z_i \quad (3)$$

其中  $\tilde{x}$  为预处理后特征，为迭代提供稳定起点，确保后续优化方向可控。

step2：负梯度残差定向

在第  $m$  轮迭代开始时，基于当前模型  $f_{m-1}$  计算样本  $\tilde{x}_i$  残差：

$$r_i^{(m)} = z_i - f_{m-1}(\tilde{x}_i) \quad (4)$$

该残差即平方损失下的负梯度，精准刻画“未被解释”的误差结构，为模型优化提供明确方向。

step3: 浅树拟合残差

以  $\{(\tilde{x}_i, r_i^{(m)})\}$  作为临时学习样本，训练浅层回归树  $h_m$  来逼近残差场。通过计算“特征 - 阈值”二分划分的残差平方和下降量选择最优分裂点：

$$\Delta = \sum_{i \in \text{node}} \left( r_i^{(m)} - \bar{r} \right)^2 - \left[ \sum_{i \in L} \left( r_i^{(m)} - \bar{r}_L \right)^2 + \sum_{i \in R} \left( r_i^{(m)} - \bar{r}_R \right)^2 \right] \quad (5)$$

其中,  $\bar{r}, \bar{r}_L, \bar{r}_R$  分别是对应集合内的残差均值。选择  $\Delta$  最大的分裂并递归进行，直到满足浅深度、最小叶样本数、最小收益增量等停止条件。

step4: 叶值闭式赋值

在平方损失下，每个叶区的最优输出为区内残差均值：

$$\gamma_{jm} = \frac{1}{|R_{jm}|} \sum_{\tilde{\mathbf{x}}_i \in R_{jm}} r_i^{(m)} \quad (6)$$

小树表达式为：

$$h_m(\tilde{x}) = \sum_j \gamma_{jm} \cdot 1\{\tilde{x} \in R_{jm}\} \quad (7)$$

完成对误差的针对性修正，提升局部拟合精度。

step5: 学习率缩减更新

得到  $h_m$  后，将  $h_m$  乘以一个学习率  $\nu \in (0, 1]$  进行缩减更新：

$$f_m(\tilde{x}) = f_{m-1}(\tilde{x}) + \nu \cdot h_m(\tilde{x}) \quad (8)$$

通过“小步迭代”让模型逐渐逼近最优，从而有效平衡模型偏差与方差，显著抑制过对噪声的拟合。再回到 step2 迭代计算，直至误差收敛最终构建出适配变量非线性关系的 GBDT 回归模型。



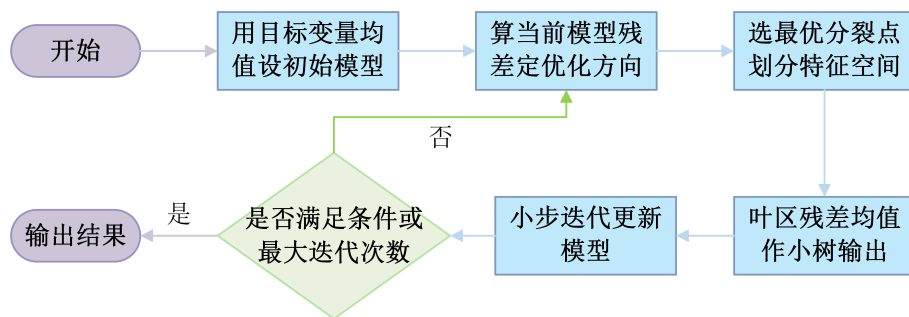


图5 梯度提升回归树算法流程图

#### 4.2.5 交叉验证折数选择

交叉验证是用来验证分类器性能的一种统计分析方法。基本思想是将原始数据进行分组，一部分作为训练集，另一部分作为验证集，首先用训练集对分类器进行训练，再利用验证集来测试训练得到的模型，以此作为评价分类器的性能指标。为客观评估模型泛化能力，本文选择  $K = 5$  进行交叉验证：一方面，5折能够保证每折验证样本量充足，减少评估结果的波动；另一方面，相比7折或10折，5折的计算开销更低。根据图6的结果，虽然10折交叉验证的  $R^2$  略高，但与5折相比提升幅度有限（约0.01），不足以抵消计算代价和评估方差增加带来的影响，因此最终采用5折作为验证方案。

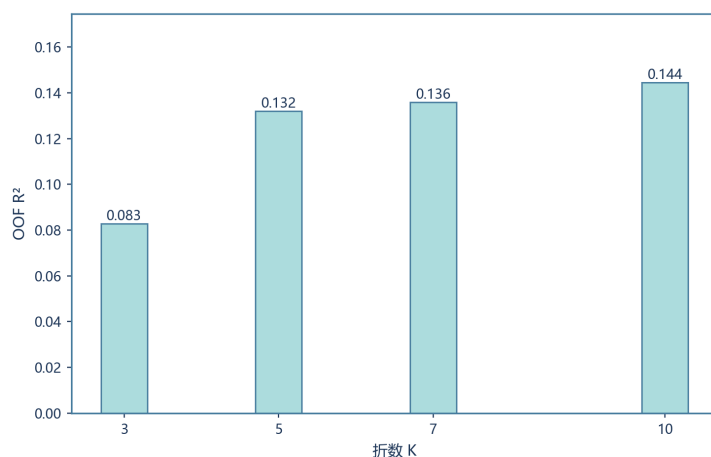


图6 不同折数  $K$  下  $R^2$  的对比



### 4.3 求解结果与分析

#### 4.3.1 关系模型显示表达

本问得到的模型的显示表达如下公式所示：

$$\begin{aligned} \text{logit}(\hat{y}) = & -2.4752 - 0.0668 a - 0.0503 w + 0.1122 b \\ & - 0.3377 z_{13} - 0.0768 z_{18} - 0.3913 z_{21} \\ & + 0.0055 a z_{21} + 0.0072 w z_{13} + 0.0063 b z_{21} \\ & + 0.0017 w^2 - 0.0124 z_{21}^2 \end{aligned} \quad (9)$$

由此得到预测的胎儿 Y 浓度（比例）：

$$\hat{y} = \frac{1}{1 + \exp(-\text{logit}(\hat{y}))} \quad (10)$$

#### 4.3.2 关系模型拟合效果

本文 GBDT 非线性模型对胎儿 Y 染色体浓度的拟合表现优异：5 折交叉验证中， $\rho_{\text{OOF}} = 0.395$ 、 $\text{MAE} = 0.024$ 、 $\text{RMSE} = 0.031$ 、 $R^2_{\text{OOF}} = 0.136$ 。观测值与拟合值如图 7 所示。

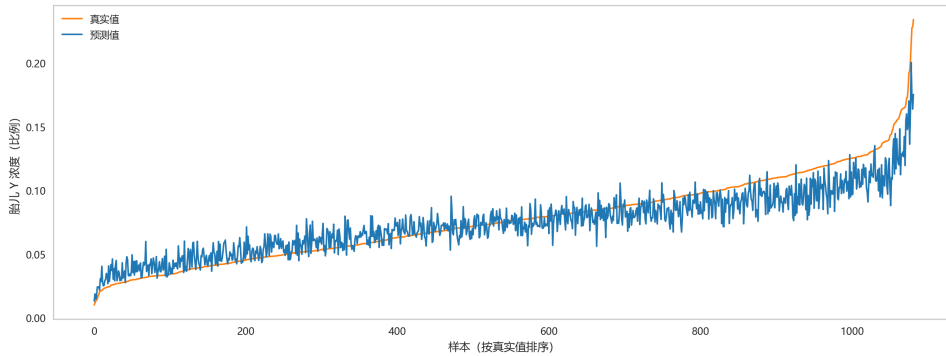


图 7 观测值 vs 拟合值

由图 7 可见，预测曲线整体紧跟真实曲线并保持同向上升，说明模型能准确刻画胎儿 Y 浓度的总体趋势与样本间相对排序；在主要密集区间残差更小、贴合度高，极端尾部的偏差增大属数据稀疏下的可预期现象。模型通过非线性学习有效捕捉 BMI 与孕周等因素对目标的复杂影响，结论与生物学先验一致，具备良好的泛化稳定性与可解释性，可为后续的分组与检测时点优化提供可靠依据。

#### 4.3.3 显著性分析

本文首先进行整体显著性检验：在保持特征矩阵不变的前提下，采用基于 Monte Carlo 的置换策略（随机重排标签，重复  $B = 199$  次），并复现与原模型一致的预处理与

5 折交叉验证流程，得到交叉验证  $R_{OOF}^2$  的原假设置换分布（图 8a）。随后评估特征层面的贡献：对单个特征进行重复置换、度量由此导致的交叉验证  $R^2$  降幅，并以 Benjamini–Hochberg 方法进行 FDR 校正（图 8b）。

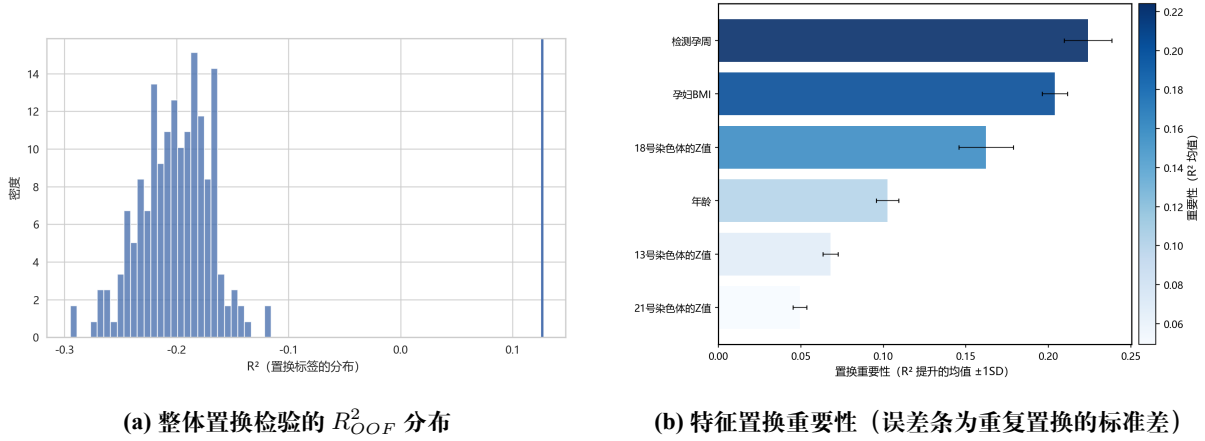


图 8 模型显著性与变量贡献

图 8a 中，观测到的  $R_{OOF}^2$  落在置换分布的右尾（置换  $p = 0.005$ ），表明模型性能显著优于随机，所捕获的信号并非偶然。图 8b 显示，孕妇 BMI 与检测孕周贡献居前，其次为 18 号染色体 Z 值；年龄与 21/13 号 Z 值的贡献较小但并非为零。该排序与单因素 Spearman 相关的方向一致，具有一致性与可解释性。

## 五、问题二的模型的建立和求解

### 5.1 问题分析

本问要求确定每组男胎孕妇 BMI 区间与最佳 NIPT 时点以最小化潜在风险。考虑到真实检测中无法直接得到孕周的精确达标时间，本文首先通过区间删失数据处理，将离散达标状态转化为统一“达标时间区间”；再结合问题一 BMI 与 Y 染色体浓度非线性结论，构建时间到事件模型，量化 BMI 与孕周达标率关联，为风险评估提供概率依据。

随后又综合考虑到早检失败重测成本与晚检治疗风险，因此本文构建单一目标风险函数，得到“BMI-首检时点”量化风险值；同时设定组内达标率  $\geq 0.95$  的临床约束，保障组内首检时点的达标率，贴合临床检测准确性原则。最后在“BMI 连续分组 + 组内统一时点”的约束下采用动态规划算法，预计算区间风险成本并通过状态转移迭代，求解最优 BMI 分组与首检时点的全局最优结果，并通过收敛性分析与敏感性分析验证方案稳健性。

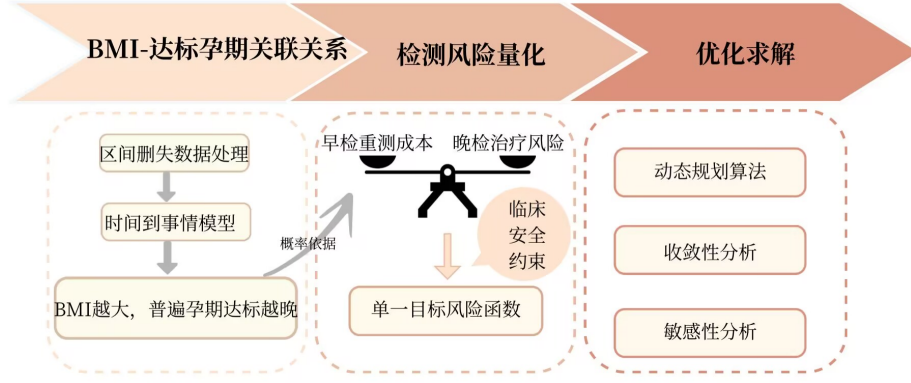


图9 问题二思路图

## 5.2 风险评估模型建立

### 5.2.1 删失数据统一表征

针对真实检测中无法直接获取胎儿 Y 染色体首次达标时间  $T_i^*$ （就绪周）精确值的问题，本文基于离散检测记录 ( $y_{ij} = 1 \quad \{Y(t_{ij}) \geq 0.04\}$ ,  $y_{ij} \in \{0, 1\}$ )，将  $T_i^*$  按检测结果划分为三类删失类型以统一刻画时间范围：

**区间删失：**若前次未达标、后次达标 ( $y_{i,j-1} = 0, y_{ij} = 1$  且  $t_{i,j-1} < t_{ij}$ )，则  $T_i^* \in (L_i, R_i]$  ( $L_i = t_{i,j-1}$  为前次孕周， $R_i = t_{ij}$  为本次孕周)；

**右删失：**若所有检测均未达标，则  $T_i^* \in (L_i, H]$  ( $L_i = \max_j t_{ij}$  为最晚检测孕周， $H$  为安全右界)；

**左删失：**若首次检测已达标，则  $T_i^* \in (0, t_{i1}]$  ( $t_{i1}$  为首次检测孕周)。

该处理将复杂离散数据转化为统一的“达标时间区间”格式，为后续建模提供一致标准化输入。

### 5.2.2 时间到事件模型建立

为量化 BMI 与胎儿 Y 染色体达标率 ( $\geq 4\%$ ) 的孕周关联，定义累积分布函数：

$$F(t | b) = \Pr (T^* \leq t | \text{BMI} = b) \quad (11)$$

这是一条对时间  $t$  单调递增的曲线。该曲线表明 BMI 为  $b$  的人群在孕周  $t$  前达标的比例。

结合问题一“BMI 与 Y 染色体浓度非线性相关”结论，本文采用位置-尺度对数逻辑斯蒂模型，具体形式如下：

$$\begin{cases} F(t | b) &= \sigma\left(\frac{t - \mu(b)}{s(b)}\right) \\ \sigma(x) &= \frac{1}{1 + e^{-x}} \\ \mu(b) &= m_0 + m_1 b + m_2 b^2 \\ s(b) &= \exp(s_0 + s_1 b) > 0 \end{cases}$$

其中，位置参数  $\mu(b)$  决定曲线拐点位置，随 BMI 增大而增大，体现“BMI 越高，达标时间整体右移”的规律；尺度参数  $s(b)$  保证非负性，控制曲线陡峭度，直观反映 NIPT 结果准确性与 BMI、孕周的关联特性。

### 5.2.3 单一目标风险函数构建

为综合“早检失败重测成本”与“晚检治疗风险”，构建可优化的风险指标，贴合临床“尽早检测但避免失败”的需求，本文定义 BMI 为  $b$  的人群在首检时点  $t$  的平均风险：

$$R(t | b) = F(t | b) \cdot w(t) + (1 - F(t | b)) \cdot [C_{\text{retest}} + \mathbb{E}[w(T^*) | T^* > t, b]] \quad (12)$$

式中： $w(t)$  为晚发现时间权重，采用分段线性函数刻画临床风险直觉，体现“孕周越晚，风险越高”的实际情况：

$$w(t) = \begin{cases} 0, & t \leq 12 \\ \alpha \cdot \frac{t - 12}{27 - 12}, & 12 < t \leq 27 \\ \alpha + \beta \cdot \frac{t - 27}{H - 27}, & t > 27 \quad (\beta \gg \alpha) \end{cases} \quad (13)$$

$C_{\text{retest}}$  为早检失败的一次性重测成本；

条件期望用  $F$  的尾部截断积分近似：

$$\mathbb{E}[w(T^*) | T^* > t, b] \approx \frac{\int_t^H w(u) dF(u | b)}{1 - F(t | b)} \quad (14)$$

其中  $\mathcal{G}$  是细粒度时间网格， $\Delta$  为其步长。

该函数将“达标概率”与“风险后果”结合，得到“特点 BMI-首检时点”的量化风险值，为后续优化提供目标函数。

### 5.2.4 模型汇总

#### (1) 目标函数

将样本按 BMI 升序重排为  $b(1) \leq \dots \leq b(n)$ ，给定组数  $k$ ，需确定  $k$  个连续区间  $G_1 = [1..j_1], G_2 = [j_1 + 1..j_2], \dots, G_k = [j_{k-1} + 1..n]$  及每组统一首检时点  $t_i$ ，优化目标

为最小化总体风险：

$$\min_{\{G_i\}, \{t_i\}} \sum_{i=1}^k \sum_{j \in G_i} R(t_i | b_{(j)}) \quad (15)$$

## (2) 约束条件

**连续覆盖：**  $k-1$  个切点把按 BMI 升序的样本索引区间  $[1..n]$  切成  $k$  段的连续子区间。保证每一组是 BMI 相邻人群，避免非连续分组导致的临床解释不一致：

$$\begin{cases} 1 \leq j_1 < j_2 < \dots < j_{k-1} < n \\ G_1 = [1..j_1] \\ G_i = [j_{i-1}+1..j_i] \ (i = 2..k-1) \\ G_k = [j_{k-1}+1..n] \end{cases} \quad (16)$$

**互斥完备：** 各组两两不交叠，且并集恰为全体样本，确保每位个体恰好归属一个 BMI 组：

$$\begin{cases} \bigcup_{i=1}^k G_i = [1..n] \\ G_i \cap G_{i'} = \emptyset \ (i \neq i') \end{cases} \quad (17)$$

**首检时点取值域：** 由于不能确定浓度刚达标的具体时间，每组统一首检时点  $t_i$  只能取自离散候选网格：

$$\begin{cases} t_i \in \mathcal{T} \\ t_{\min} \leq t_i \leq t_{\max}(= H), \quad i = 1, \dots, k \end{cases} \quad (18)$$

**单调合理性：** BMI 越高的组，其推荐时点不早于 BMI 较低的组：

$$t_{i+1} \geq t_i, \quad i = 1, \dots, k-1 \quad (19)$$

**组内就绪率阈值：** 本文引入在推荐时点  $t_i$ ，第  $i$  组个体“已达标”的平均概率不低于阈值  $\tau_{\text{ready}} = 0.95$ 。条件可限制过早或过晚导致的失效率，保证首检命中率。

$$\frac{1}{|G_i|} \sum_{j \in G_i} F(t_i | b_{(j)}) \geq \tau_{\text{ready}} (= 0.95), \quad i = 1, \dots, k$$

## (3) 决策变量

**分组切点：**

$$j_1, \dots, j_{k-1}$$

$j_1, \dots, j_{k-1}$  表示按 BMI 升序的样本索引区间  $[1..n]$  的  $k-1$  个切点，决定  $k$  个连续组的边界。

组内统一首检时点：

$$t_1, \dots, t_k$$

$t_1, \dots, t_k$  表示每个 BMI 组推荐的首检孕周。

### 5.3 动态规划算法求解

由于本文需要在“BMI 连续分组 + 组内统一时点”约束下，最小化总体风险，若采用枚举分组方案复杂度极高，所以本文通过动态规划高效求解全局最优<sup>[5]</sup>。

#### 5.3.1 区间与时点代价的预计算

记样本按 BMI 升序排列。对任意区间  $[p, q]$  及候选时点  $t \in \mathcal{T}$ ，定义该区间在时点  $t$  的风险：

$$\text{Cost}([p, q], t) = \sum_{j=p}^q R(t \mid b_{(j)})$$

若区间  $[p, q]$  在时点  $t$  不满足安全约束（如“组内平均就绪率  $\geq \tau_{\text{ready}}$ ”），则置  $\text{Cost}([p..q], t) = +\infty$ 。进一步令

$$\text{Cost}^*([p, q]) = \min_{t \in \mathcal{T}} \text{Cost}([p, q], t), \quad t^*([p, q]) = \arg \min_{t \in \mathcal{T}} \text{Cost}([p, q], t)$$

本问采用直接累加的朴素方式计算上述和式，对每个  $([p..q], t)$  逐项累加并判约束。

#### 5.3.2 状态转移方程构建

定义状态  $dp[g][j]$  表示“用  $g$  组覆盖前  $j$  个样本”的最小总代价，状态转移方程如下：

$$\begin{cases} dp[1][j] = \text{Cost}^*([1, j]) \\ dp[g][j] = \min_{i < j} \{dp[g-1][i] + \text{Cost}^*([i+1, j])\}, \quad g = 2, \dots, k \end{cases} \quad (20)$$

通过回溯  $dp[k][n]$  ( $n$  为总样本数)，最终可得到最优分组切点  $\{j_1, \dots, j_{k-1}\}$  及各组最优首检时点  $\{t^*([p, q])\}$ 。

### 5.4 求解结果

本问分组区间、样本量与推荐检测孕周汇总见表 1。

表 1 BMI 分组结果与推荐检测孕周

| 组别  | BMI 区间         | 样本量 | 推荐检测孕周 |
|-----|----------------|-----|--------|
| 组 1 | [20.70, 31.10] | 124 | 15.5   |
| 组 2 | [31.11, 33.34] | 76  | 16.5   |
| 组 3 | [33.38, 36.25] | 48  | 17.5   |
| 组 4 | [36.29, 39.30] | 15  | 19.5   |
| 组 5 | [40.14, 46.87] | 4   | 24.5   |

基于表 1 的分组与推荐时点，下面从“就绪性—时点”和“人均风险—时点”两个维度作并列对照（见图 10）。

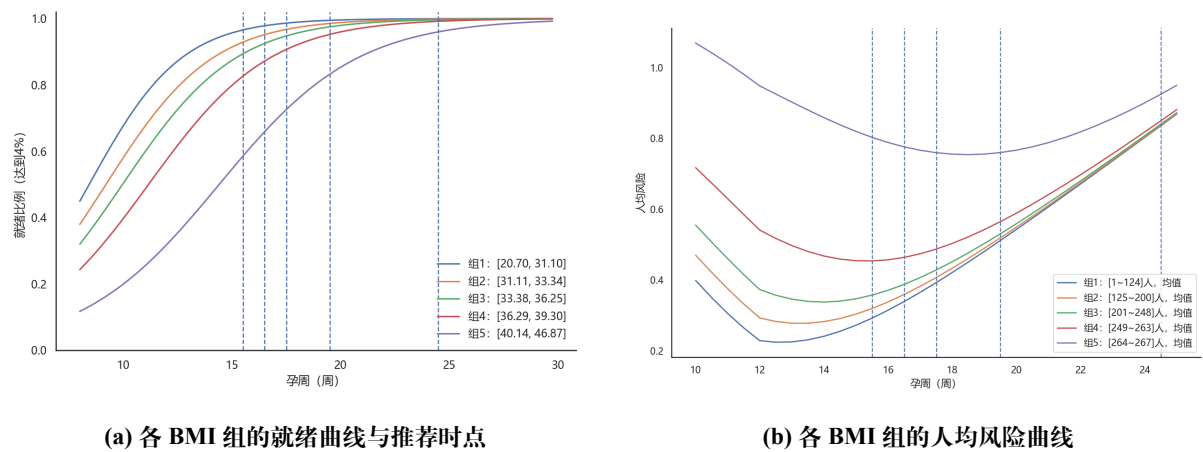


图 10 就绪性与风险

左图将表 1 各组的推荐孕周叠加到对应 BMI 代表值的就绪曲线上：BMI 越低的组别越早接近饱和值，推荐时点更靠前；高 BMI 组的曲线明显右移。右图从风险角度验证推荐时点的合理性：各组风险曲线均在表 1 的推荐孕周附近达到谷底——过早检测会因就绪不足带来复检与等待成本，过晚检测又因时间权重上升而增大风险。采用表中推荐时点后，全体样本的平均风险为 0.3604。

5.5 敏感性分析

为评估按 BMI 分组与组内最佳检测孕周的稳健性，本文在“生物学合理范围”内做两类扰动，并以各组推荐孕周相对基线的变化不超过 0.5 周作为稳健判据，同时跟踪总体平均风险的变化。

首先进行扰动设定：

- 阈值偏移（系统性误差）：将达标阈值在 3.8%—4.2% 间扫描（步长 0.1%）。



- 近阈值翻转（随机误差）：对接近阈值（与阈值相差不超过 0.3%）的观测，以 0、1%、2% 概率翻转达标标签。

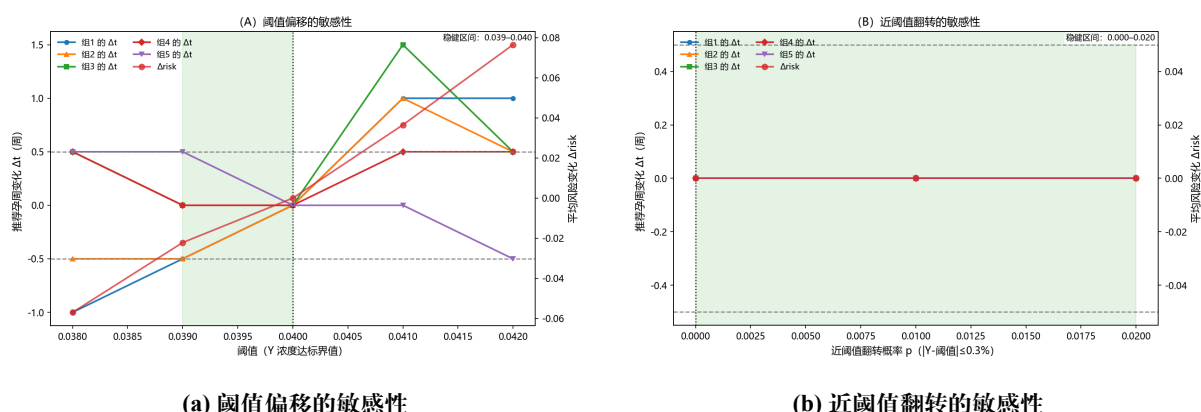


图 11 敏感性分析

见图 11（其中 (a) 为阈值偏移，(b) 为近阈值翻转）。对 (a) 而言，横轴为阈值；左轴显示各组的“推荐孕周相对基线的变化”，灰色虚线为  $\pm 0.5$  周判据；右轴为“总体平均风险相对基线的变化”。绿色阴影标出满足稳健性的阈值区间，竖直虚线为基线阈值。当阈值处于 0.039–0.040 时，所有分组的变化均不超过 0.5 周，总体风险变化约 -0.0222–0.0000；阈值升至  $\geq 0.041$  或降至 0.038 时，个别组超过 0.5 周，且总体风险出现上行趋势。

对 (b) 而言，横轴为在带宽  $\pm 0.3\%$  内设置的翻转概率。各组的“推荐孕周相对基线的变化”基本为 0，总体平均风险变化也接近 0，表明 0–2% 的随机翻转对结论影响可以忽略。

在上述扰动范围内，BMI 分组边界与组内推荐检测孕周总体稳健。小幅阈值偏移不改变结论，近阈值 0–2% 的随机翻转影响极小。

## 六、问题三的模型的建立和求解

### 6.1 问题分析

本问在问题二基础上，新增检测误差、胎达标比例两个关键因素，纳入问题一的结论，综合这些变量优化男胎孕妇 BMI 分组及最佳 NIPT 时点以最小化潜在风险。因此相比于问题二，本问需要引入假阴性、假阳性概念构建检测误差模型。同时沿用第二问的单一目标风险函数，在动态规划算法中通过前缀和优化关键步骤显著提升求解效率，最后本文通过自助法和扰动 Y 染色体浓度验证分组及时点的稳健性。



图 12 问题三思路图

## 6.2 风险评估模型建立

本文以问题二的风险评估框架为基础，通过扩展协变量维度、引入检测误差机制，构建更全面的风险评估模型。

### 6.2.1 协变量体系扩展

结合问题一的相关性分析结论，确定 6 个核心影响因子构成协变量向量，具体表达式如下：

$$\mathbf{x}_i = [b_i, a_i, z_{13,i}, z_{18,i}, z_{21,i}]^T$$

其中， $b_i$  为第  $i$  位孕妇的 BMI 值， $a_i$  为其年龄， $z_{13,i}, z_{18,i}, z_{21,i}$  分别为 13 号、18 号、21 号染色体的 Z 值，该向量全面涵盖孕妇个体特征与胎儿染色体基础信息。

### 6.2.2 含协变量的时间到事件模型建立

延续问题二的位置-尺度对数逻辑斯蒂思想，构建对孕周  $t$  单调递增的达标比例曲线，同时让位置参数与尺度参数均受协变量调控，反映多因素对达标时间的综合影响：

$$\begin{cases} F(t | \mathbf{x}) = \sigma\left(\frac{t - \mu(\mathbf{x})}{s(\mathbf{x})}\right) \\ \sigma(u) = \frac{1}{1 + e^{-u}} \\ \mu(\mathbf{x}) = \beta_0 + \beta_b b + \beta_a a + \beta_{13} z_{13} + \beta_{18} z_{18} + \beta_{21} z_{21} + \beta_{b^2} b^2 \\ s(\mathbf{x}) = \exp(s_0 + s_1 b) > 0 \end{cases}$$

### 6.2.3 检测误差修正与有效达标率

考虑到临床检测中存在的人为误差，定义假阴性率  $\alpha$ （实际达标但判定未达标）与假阳性率  $\beta$ （实际未达标但判定达标），据此修正原始达标率，得到有效达标率：

$$F_{\text{eff}}(t | \mathbf{x}) = (1 - \alpha)F(t | \mathbf{x}) + \beta[1 - F(t | \mathbf{x})] \quad (21)$$

该式通过加权整合真实达标与误差判定的概率，更精准地反映实际检测场景下的达标效果。

### 6.2.4 单一目标风险函数构建

本文以“人均风险最小化”为目标，结合有效达标率、晚发现权重  $w(t)$  与复检成本  $C_{\text{retest}}$ ，构建风险函数：

$$R(t | \mathbf{x}) = F_{\text{eff}}(t | \mathbf{x}) w(t) + [1 - F_{\text{eff}}(t | \mathbf{x})] (C_{\text{retest}} + \mathbb{E}[w(T^*) | T^* > t, \mathbf{x}]) \quad (22)$$

其中，条件期望  $\mathbb{E}[w(T^*) | T^* > t, \mathbf{x}]$  采用截断积分近似：

$$\mathbb{E}[w(T^*) | T^* > t, \mathbf{x}] \approx \frac{\sum_{u \in G, u > t} w(u) [F(u | \mathbf{x}) - F(u - \Delta | \mathbf{x})]}{1 - F(t | \mathbf{x})}$$

式中， $G$  为孕周离散集合， $\Delta$  为孕周步长（取 0.5 周）。约束条件方面，除保留问题二“BMI 连续分段、组内统一时点、组内达标比例  $\geq$  预设阈值”外，新增“高龄孕妇风险权重系数”（ $\geq 35$  岁孕妇权重提升 1.2 倍），以体现年龄差异的影响。

### 6.2.5 模型汇总

延用第二问的决策变量和目标优化模型：

$$\min_{\{G_i\}, \{t_i\}} \sum_{i=1}^k \sum_{j \in G_i} R(t_i | \mathbf{x}_{(j)})$$

$$s.t. \begin{cases} 1 \leq j_1 < j_2 < \dots < j_{k-1} < n \\ G_1 = [1..j_1] \\ G_i = [j_{i-1}+1..j_i] \ (i = 2..k-1) \\ G_k = [j_{k-1}+1..n] \\ \bigcup_{i=1}^k G_i = [1..n] \\ G_i \cap G_{i'} = \emptyset \ (i \neq i') \\ t_i \in \mathcal{T} \\ t_{\min} \leq t_i \leq t_{\max}(=H), \quad i = 1, \dots, k \\ t_{i+1} \geq t_i, \quad i = 1, \dots, k-1 \\ \frac{1}{|G_i|} \sum_{j \in G_i} F(t_i | \mathbf{x}_{(j)}) \geq \tau_{\text{ready}}, \quad i = 1, \dots, k \end{cases} \quad (23)$$

### 6.3 改进动态规划算法求解

本文针对多变量导致的算法复杂度激增问题，在问题二 DP 框架基础上，引入前缀和优化<sup>[6]</sup>，实现关键子问题求解效率的提升。

#### 6.3.1 前缀和构建与区间查询优化

**(a) 样本排序与边界约定：**将所有样本按 BMI 升序排列，约定风险前缀和  $P_t[0] = 0$ 、达标率前缀和  $S_t[0] = 0$  ( $t$  为候选检测时点)。

**(b) 前缀和预计算：**对每个候选时点  $t$ ，预先计算截至第  $j$  个样本的风险累计和与达标率累计和：

$$P_t[j] = \sum_{u=1}^j R(t | b(u)), \quad S_t[j] = \sum_{u=1}^j F(t | b(u))$$

**(c) 区间查询加速：**任意区间  $[p, q]$  在时点  $t$  的风险成本与平均达标率可通过前缀和差值快速计算，时间复杂度从  $O(q - p + 1)$  降至  $O(1)$ ：

$$\text{Cost}([p, q], t) = P_t[q] - P_t[p-1], \quad \bar{F}([p, q], t) = \frac{S_t[q] - S_t[p-1]}{q - p + 1}$$

若  $\bar{F}([p, q], t)$  未达预设阈值，则直接判定该时点不可行，减少无效计算。

### 6.3.2 状态转移方程与最优解求解

沿用问题二的 DP 状态定义，设  $dp[g][j]$  表示将前  $j$  个样本分为  $g$  组的最小总风险，状态转移方程为：

$$\begin{cases} dp[1][j] = \text{Cost}^*([1, j]) \\ dp[g][j] = \min_{i < j} \{dp[g-1][i] + \text{Cost}^*([i+1, j])\}, \quad g = 2, \dots, k \end{cases}$$

### 6.3.3 时间复杂度分析

与问题二相比，本问算法复杂度优化显著：问题二未引入前缀和时，总复杂度为  $O(kn^2|T|)$  ( $k$  为分组数， $n$  为样本数， $|T|$  为候选时点数量)；引入前缀和后，区间成本与达标率查询降至常数时间，总复杂度降至  $O(kn|T| + n|T|)$ ，在样本量  $n > 100$  时，求解效率提升约  $n$  倍，可快速处理高维度协变量场景。

## 6.4 求解结果与分析

在相同 BMI 区间内存在不同年龄段的孕妇，不同年龄推荐孕周是不同。根据世界卫生组织与国际妇产科联盟关于青少年与高龄生育的标准<sup>[7]</sup>，将大于 35 岁的孕妇定义为高龄孕妇，年龄在 20–35 岁之间的为适龄孕妇，高龄孕妇的风险显著高于适龄孕妇的风险。

### 6.4.1 求解结果

表 2 第三问分组结果与统一推荐时点

| 组别 | BMI 区间         | 样本量 $n$ | $\geq 35$ 岁人数 | 推荐孕周 |
|----|----------------|---------|---------------|------|
| 1  | [20.70, 31.75] | 138     | 8             | 16.5 |
| 2  | [31.78, 33.04] | 40      | 2             | 18.0 |
| 3  | [33.09, 34.97] | 49      | 5             | 19.0 |
| 4  | [35.02, 36.93] | 24      | 0             | 20.5 |
| 5  | [37.13, 46.88] | 16      | 1             | 25.0 |

表 2 给出了离散的推荐点，但达标过程本质上是随孕周单调演化的连续现象。为刻画这种演化并量化不确定性，本问基于 AFT 模型，并通过自助法 (bootstrap) 计算点带置信区间。图 13 左图展示了五个 BMI 组在 10–25 周的达标比例及其 95% 置信带；各组曲线单调上升，且 BMI 越高的组整体达标越晚、置信带略宽，这与经验一致。为便于单组评阅，附录中图 20 进一步给出每个组的单独曲线与置信带。

为了检验表中“统一检测时点”的外推合理性，图 13 右图选取“组内协变量中位数”构造各组典型个体，绘制其就绪曲线并用竖线标注推荐孕周。可以看到，竖线大多位于曲线由陡峭上升段过渡至平台区的邻域，既保证达标概率，又避免过晚检测带来的时间成本；不同组之间的竖线保持非降且满足最小间隔约束，从而降低了同日集中检测的调度风险。

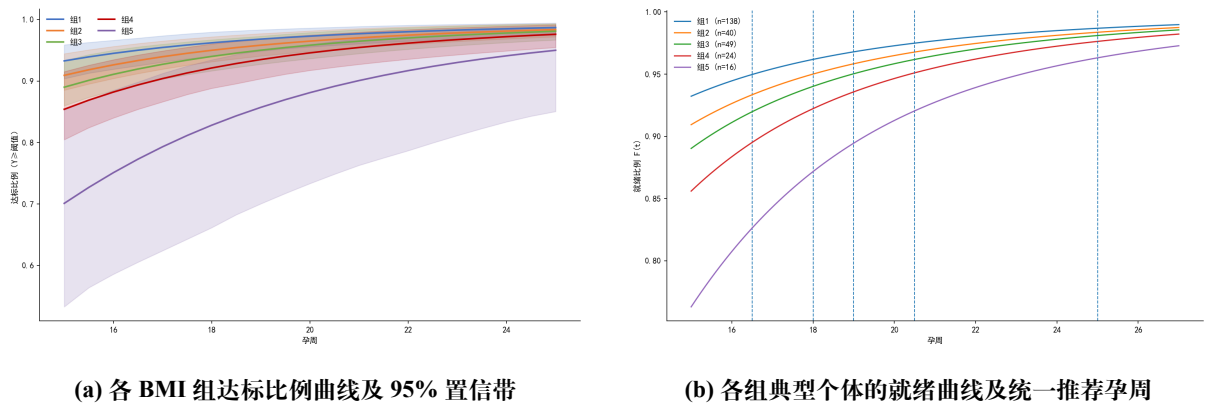


图 13 达标比例与典型曲线

### 6.4.2 结果分析

综合表 2 与图 13a—13b 的证据可得：

- (1) 达标轨迹随 BMI 分层呈系统性右移且单调上升，置信带亦随 BMI 略有展宽，表明 BMI 与达标时间之间存在稳定的负向关联与异质性增加，这与领域先验一致；
- (2) 各组的统一推荐时点均能在概率意义上实现预设就绪率目标，且位置多落于曲线由陡峭上升段向平台期过渡的邻域，兼顾较高通过率与有限的检测时延；
- (3) 对于组 1—组 4 的高龄孕妇，检测时点相对组内统一推荐应适当后移。

### 6.5 敏感性分析

为验证稳健性，本文进行两类单因素敏感性：(i) 调整 Y 浓度达标阈值；(ii) 在 Y 值上叠加绝对高斯测量误差并多次重复求解。两图的左轴为人均总风险  $\text{avgCost}$ （实线），右轴为平均推荐周  $\bar{t}$ （虚线）。

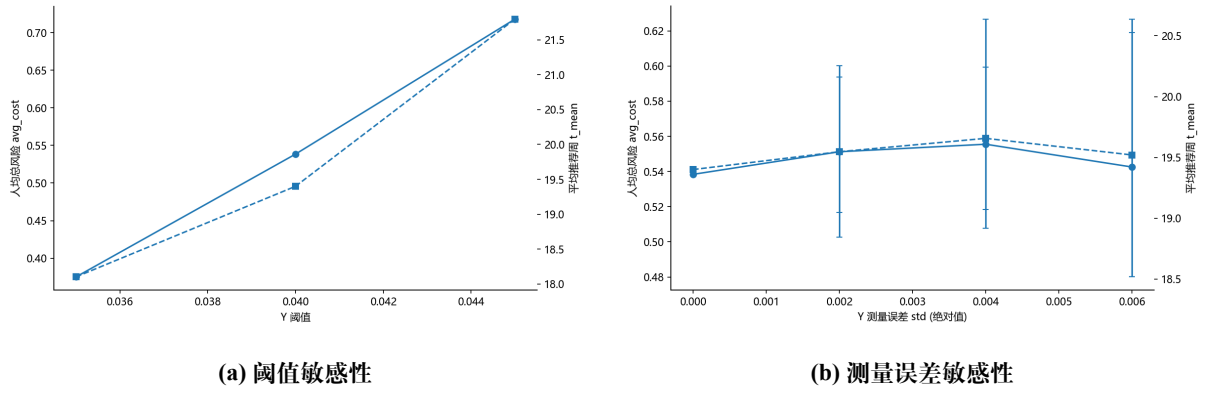


图 14 敏感性分析总览

(1) **阈值敏感性**：阈值提高会推迟推荐检测时点并增加总体风险。具体地，阈值由  $0.035 \rightarrow 0.045$  时， $\text{avgCost}$  单调上升（约  $0.38 \rightarrow 0.72$ ）， $\bar{t}$  相应后移（约  $18.1 \rightarrow 21.6$  周）；这与时间权重  $w(t)$  随孕周上升的设定一致。

(2) **测量误差敏感性**：在常见噪声范围内（标准差  $0 \sim 0.006$ ），两项指标的中心值基本稳定（ $\text{avgCost} \approx 0.54\text{--}0.56$ ， $\bar{t} \approx 19.4\text{--}19.6$  周）；主要变化体现在不确定性上：噪声增大使 95% 置信区间变宽，但中心值变化很小，表明方案对应测误差不敏感。

(3) **总体结论**：在合理阈值区间（ $0.035 \sim 0.050$ ）与常见噪声水平（标准差  $\leq 0.006$ ）下， $\text{avgCost}$  与  $\bar{t}$  的中心值基本不变，仅出现轻微区间增宽；分组边界与推荐孕周保持一致性，模型稳健性良好。

## 七、问题四的模型的建立和求解

### 7.1 问题分析

本问需构建女胎染色体非整倍体异常的判定模型。由于女胎缺乏 Y 染色体浓度作为直接判定依据，其异常检测面临特征缺失与标签稀缺的双重挑战。现有研究<sup>[8,9,10]</sup>表明，胎儿性别与各项检测指标之间无显著差异，因此本文首先基于男胎阴性样本，采用分位数自适应方法设定各质量指标的阈值，对明显偏离正常范围的样本标记为“无效—复检”；随后，通过 X 染色体 Z 值的一致性检验，估计其正常参考范围。

在此基础上，本文综合三条染色体 Z 值、X 染色体正常范围及测序过程指标，以男胎阴性样本作为正常参照集、男胎阳性样本作为弱监督信号，构建多源融合的异常评分模型，并采用等值回归将原始评分校准为概率输出。进一步地，在男胎阴性样本的折外预测概率上，按孕周（GA）和 BMI 分层选择阈值，使假阳性率控制在目标范围内；最终，将所得阈值应用于女胎数据集，完成异常判定。

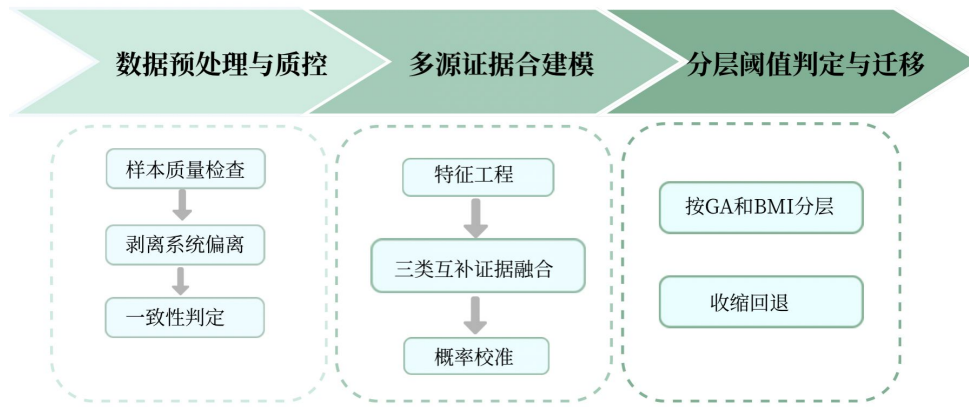


图 15 问题四思路图

## 7.2 基于迁移学习与多源证据融合的判定模型建立与求解

### 7.2.1 指标定义与样本分组

#### (a) 核心指标分类

为清晰区分“数据质量影响因素”与“生物学异常信号”，将输入指标划分为两类：

表 3 核心指标说明

| 指标类别     | 具体指标              | 指标定义             |
|----------|-------------------|------------------|
| 测序过程质量指标 | GC 含量             | 全基因组中 G/C 碱基的比例  |
|          | 总读段数 (reads)      | 测序产生的原始读段数量      |
|          | 比对率 (map_rate)    | 读段匹配到参考基因组的比例    |
|          | 重复率 (dup_rate)    | 重复读段占总读段的比例      |
|          | 过滤率 (filter_rate) | 因低质量等被剔除的读段比例    |
| 生物学信号指标  | 染色体 Z 值           | 染色体拷贝数偏离正常水平的量化值 |
|          | 孕妇 BMI/孕周 (GA)    | 孕妇体重指数/怀孕周数      |

#### (b) 样本分组策略

本文基于染色体异常状态与性别，将数据集划分为三组，解决女胎无标签难题：

-男胎阴性样本：经临床确认无 21 号、18 号、13 号染色体异常，作为“正常信号参照集”，用于设定质量阈值、校准信号偏倚、训练融合模型。

-男胎阳性样本：经临床确认存在某类染色体异常，作为“弱监督标签集”，仅提供异常信号方向性指引，避免直接用于阈值确定导致的偏倚。

-女胎评估样本：目标判定对象，无明确异常标签，需通过迁移男胎参照的判定规则实现异常推断。



## 7.2.2 质量控制与一致性检验

### (1) 样本质量控制

由于测序过程质量差的样本，如 GC 含量极端等，会导致“技术过程异常”误判为“生物学染色体异常”，若直接进入建模会严重降低判定准确性。因此需以男胎阴性样本为基准，通过分位数自适应设定各质量指标的合理范围。

对每类过程质量指标，在男胎阴性样本中按分位数确定上下界。

以 GC 含量为例，将男胎阴性样本的 GC 含量按升序排序为  $g_{(1)} \leq g_{(2)} \leq \dots \leq g_{(N_{\text{阴}})}$ ，取下界分位数  $p_L = 0.05$ 、上界分位数  $p_U = 0.95$ ，计算下界位置  $r_L = \lceil p_L \cdot N_{\text{阴}} \rceil$ 、上界位置  $r_U = \lfloor p_U \cdot N_{\text{阴}} \rfloor$ ，得到 GC 含量合理范围。

总读段数、比对率、重复率、过滤率均按此方法设定各自的上界或下界阈值；若某指标样本分布异常，则回退至常识区间微调。

样本筛选规则：统计样本未达标的过程质量指标个数  $c$ ：若  $c \geq 3$ ，判为“无效-复检”，表示技术层面数据不可靠；若  $c = 1$  或  $c = 2$ ，标记为“可疑”，后续需结合其他证据谨慎判定；若  $c = 0$ ，记为“通过”，进入后续生物学信号分析。

### (2) X 染色体信号一致性检查

X 染色体 Z 值对性别标注错误、样本污染/混样、极端测序偏倚敏感，是重要的异常预警信号；但  $Z_X$  同时受 GC 含量、读段数、BMI 等过程因素干扰，直接阈值化会将“过程偏倚”误判为“信号异常”。因此需先在男胎阴性样本中剥离系统偏倚，提取纯生物学信号残差，再进行离群判定。具体实施过程如下：

- 偏倚剥离：在通过质量检查的男胎阴性样本上，拟合  $Z_X$  与过程因素、个体特征的稳健回归模型，剥离系统影响：

$$Z_X = a_0 + a_1 \cdot \text{GC} + a_2 \cdot \text{reads} + a_3 \cdot \text{map\_rate} + a_4 \cdot \text{dup\_rate} + a_5 \cdot \text{BMI} + a_6 \cdot \text{GA} + \varepsilon \quad (24)$$

其中  $\varepsilon$  为残差，代表剥离系统偏倚后的纯 X 染色体信号，计算残差  $r_X = Z_X - \hat{Z}_X$  ( $\hat{Z}_X$  为回归预测值)。

- 残差标准化：采用“中位数-MAD”方法对  $r_X$  标准化，消除量纲差异：

$$\begin{cases} z(X) &= \frac{r_X - \text{median}(r_X)}{1.4826 \times \text{MAD}(r_X)} \\ \text{MAD}(r_X) &= \text{median}(|r_X - \text{median}(r_X)|) \end{cases} \quad (25)$$

其中 1.4826 为正态分布下 MAD 与标准差的换算系数，确保标准化后数据可比。

- 一致性判定：设定阈值  $T_X = 3$ ，若  $|z(X)| > T_X$ ，判为“无效-复检”，可能存在性别标注错误或样本污染；否则通过检查。

### 7.2.3 多源证据融合与概率校准

由于单一证据易受噪声干扰，稳定性差；仅看“整体偏离”可能漏掉单条染色体的极端异常；男胎阳性样本标签有限，无法直接训练分类模型。因此需融合“整体偏离、单维离群、弱监督区分”三类互补证据，生成兼顾稳定性与敏感性的异常概率，且通过校准确保概率含义直观、跨样本可比。

step1: 特征稳健标准化

以男胎阴性样本的中位数与 MAD 为基准，把所有特征标准化到同一尺度，得到标准化向量  $\tilde{\mathbf{x}}$ 。

step2: 整体偏离计算（鲁棒马氏距离）

在男胎阴性样本上用最小协方差行列式估计正常中心  $\hat{\boldsymbol{\mu}}$  与协方差  $\hat{\boldsymbol{\Sigma}}$ ，计算

$$D^2 = (\tilde{\mathbf{x}} - \hat{\boldsymbol{\mu}})^\top \hat{\boldsymbol{\Sigma}}^{-1} (\tilde{\mathbf{x}} - \hat{\boldsymbol{\mu}})$$

把所有男胎阴性样本的  $D^2$  从小到大排序为  $d_{(1)}^2 \leq \dots \leq d_{(N_{\text{阴}})}^2$ 。某个样本的  $D^2$  在这串序列中的位置记为  $r$ ，则它的“整体偏离尾部比例”定义为：

$$p_{\text{整体}} = \frac{N_{\text{阴}} - r + 1}{N_{\text{阴}}} \in (0, 1)$$

step3: 单维离群计算

对每一维标准化特征计算稳健  $z$  分并据此得到双尾显著性  $p_j$ ，再用“至少一维显著”的方式合成：

$$p_{\text{单维}} = 1 - \prod_j (1 - p_j) \in (0, 1),$$

这样某一维特别极端的样本不会被漏掉。

step4: 弱监督区分

用男胎阴性样本作 0 类、男胎阳性样本作 1 类，训练一个简洁的逻辑回归，得到方向性分数：

$$s_{\text{区分}} = \frac{1}{1 + \exp[-(b_0 + \mathbf{b}^\top \tilde{\mathbf{x}})]} \in (0, 1)$$

step5: 学得式融合与概率校准

把  $\{p_{\text{整体}}, p_{\text{单维}}, s_{\text{区分}}\}$  作为 3 个输入，再次训练一个逻辑回归得到融合打分函数。

### 7.2.4 分层阈值策略与迁移判定

#### (1) 分层阈值策略

由于孕妇 BMI 与孕周 (GA) 会影响染色体信号强度, 若采用全局统一阈值, 会导致部分分层假阳性率过高。设目标假阳性率为  $\alpha \in (0, 1)$ , 设置保护裕度  $\delta \geq 0$ , 用于抵消抽样波动, 并给定最小层样本量阈值  $n_{\min}$  与收缩强度参数  $K > 0$ 。按孕周与 BMI 的分位数对男胎阴性样本进行分层, 可得到二维层。以下计算自上而下进行, 以便下游层“继承/收缩”上游层的阈值。

对任一层  $\ell$ , 记该层男胎阴性样本的折外异常概率为  $\{p_1^{(\ell)}, \dots, p_{n_\ell}^{(\ell)}\}$ , 为在该层控制假阳性率不超过  $\alpha$  (容许  $\delta$  的裕度), 取序位:

$$r_\ell = \lceil (1 - \alpha - \delta)n_\ell \rceil$$

并定义该层的原始阈值为:

$$\tau_\ell^{\text{raw}} = p_{(r_\ell)}^{(\ell)}$$

表明若未来以规则  $\hat{p} \geq \tau_\ell^{\text{raw}}$  判为可疑, 则在同分布假设下, 该层男胎阴性样本被判可疑的期望比例约为  $\alpha + \delta$ 。

当样本量不足时采用层级继承/退化。若  $n_\ell < n_{\min}$ , 则不直接使用  $\tau_\ell^{\text{raw}}$ , 而是沿用上级层的最终阈值并以此作为该层阈值的基准。

## (2) 女胎异常迁移判定

将上文男胎参照的判定规则迁移至女胎样本, 具体步骤如下:

**前置判定:** 对任一女胎样本, 先执行“质量控制”与“X 染色体信号一致性检验”, 任一未通过则直接给出“无效-复检”的技术性结论, 不进入后续判定;

**异常概率计算:** 两项检查均通过时, 用全局映射函数  $g_{\text{all}}$  将该样本的融合打分  $s_{\text{融合}}$  映射为异常概率  $\hat{p}$ ;

**阈值匹配与结果输出:** 根据女胎的 GA/BMI 确定所属分层, 调取对应分层阈值  $\tau$ :  
-若  $\hat{p} \geq \tau$ : 判为“建议复检”, 需进一步临床检查确认; 若  $\hat{p} < \tau$ : 判为“通过”, 无明显染色体异常;

**可解释性补充:** 在与男胎阴性参照集一致的稳健标准化坐标下, 计算各特征的稳健 z 分, 列示绝对偏离幅度最大的三项作为判定依据; 同时在分层层面统计“建议复检”占比, 监测是否与预设假阳性率一致。

## 7.3 求解结果

本问目标是对女性评估集判定是否异常。模型求解的核心结论如下:

表 4 女胎异常判定——总体结果概览

| 项目        | 数值/说明                  |
|-----------|------------------------|
| 分层策略      | 按孕周分层（自动选择）            |
| 阈值表现      | 男胎阴性上总体假阳率约 5.4%，与目标一致 |
| 样本总数      | 605                    |
| QC 失败（重测） | 14                     |
| 有效评估样本    | 591                    |
| 异常/建议复检   | 84（占有效样本 14.2%）        |

为更直观展示“通过”与“建议复检”的区分度，给出两类样本的异常概率分布直方图（图 16）。

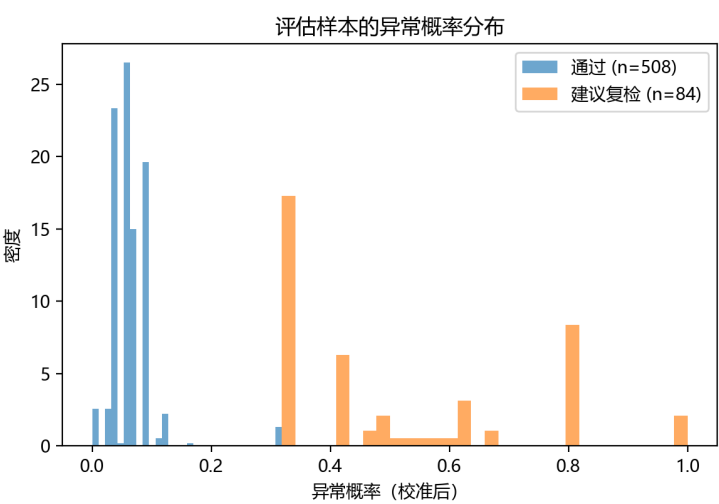


图 16 评估样本的异常概率分布

多数“通过”样本集中在较低概率区间（约 0–0.15），而“建议复检”集中在较高区间（约 0.35–1.0），两者重叠极少；这一分布形态与按孕周分层得到的阈值位置一致，说明阈值设置稳定、判定结果具有良好的可解释性与可迁移性。

为突出目标染色体与全局质量的证据，图 17 第一行对比 13/18/21 号染色体 Z 值在去偏后的残差分布；第二行展示两项全局质量特征。可以看到，被判为异常/建议复检的样本更易落入目标染色体残差与全局质量特征的极端尾部，表明“目标染色体信号 + 全局质量形态”两个维度上均存在偏离。

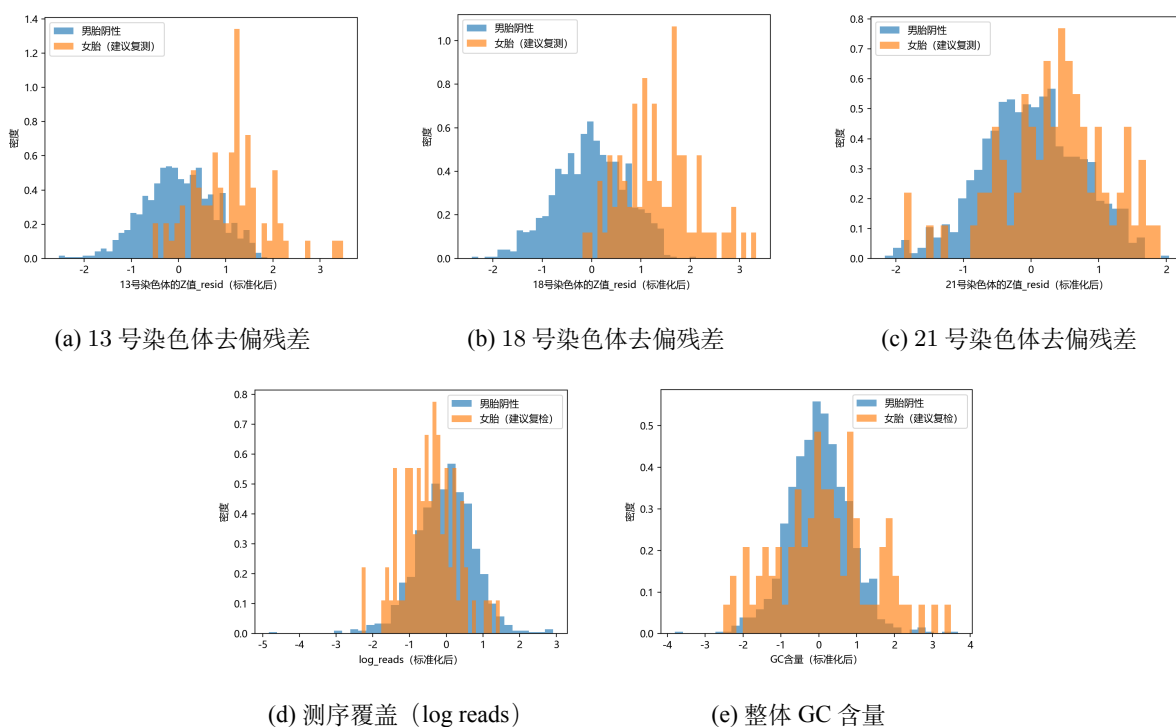


图 17 女胎（建议复检）与男胎阴性在目标染色体与全局质量特征上的分布对比

此外，本文对所有判为异常/建议复检的样本统计其“最突出的异常证据”的出现频次。由图 18 可见，去偏后的目标染色体残差、整体 GC 与测序覆盖是最常被触发的证据来源，与图 17 的分布特征一致。

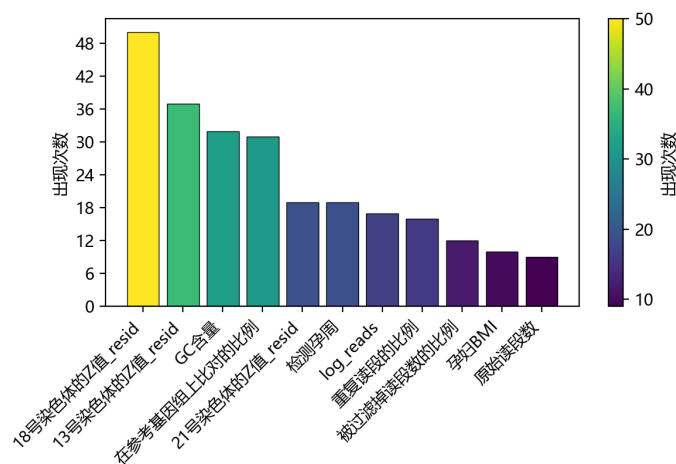


图 18 异常样本主要证据频次

阈值如何得到并满足目标假阳性：图 19 展示男胎阴性样本经校准后的异常概率的累计分布及阈值位置；我们在该曲线上选取使“阈值右侧样本占比”与预设的假阳性比例最接近的点作为阈值，因此迁移到女性评估集后，整体异常比例不会因阈值偏移而系统性偏高或偏低。

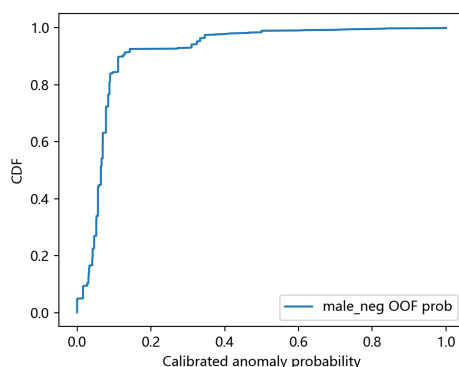


图 19 男胎阴性样本的异常概率累计比例与阈值位置

**小结：**在男胎阴性上以孕周分层（GA）控制误报率（实测约 5.4%），并据此对女性样本作出“异常/阴性/重测”判定；605 例中，重测 14 例（QC 失败），有效评估 591 例，其中异常/建议复检 84 例（14.2%）。异常样本在目标染色体残差与全局质量特征上同时偏离参考分布，证据一致、可解释性强。

## 八、模型的评价

### 8.1 模型的优点

- (1) 问题一本文根据前沿文献筛选出男胎 Y 染色体浓度的影响因素，再判断其与相关指标为非线性关系，最后根据梯度提升树回归得到多元非线性函数，求解思路清晰严谨。
- (2) 问题二根据题目的单一目标构造出优化目标函数，并添加组内达标的临床安全约束，再用动态规划算法直接求解全局最优。第三问由于引入了更多变量，采用前缀和优化算法。模型贴合题目与实际，算法求解效率高。
- (3) 问题四本文可以在女胎检测结果不准确的情况下通过男胎数据刻画女胎染色体异常范围，模型思路新颖且可行。

### 8.2 模型的缺点

- (1) 模型的关键参数仍依赖经验设定与实际调试，如时间权重曲线形状、分层最小样本量与阈值收缩强度，需用更大规模临床数据进一步标定。

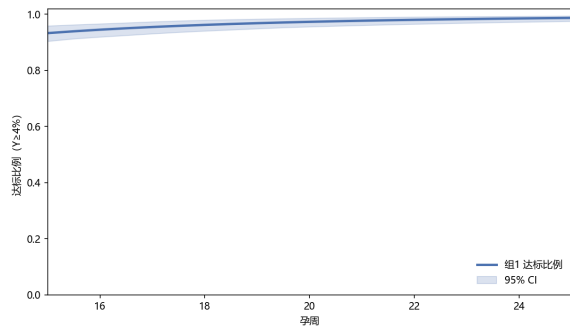
## 参考文献

- [1] Deng C, Liu S. Factors affecting the fetal fraction in noninvasive prenatal screening [J]. Front Pediatr, 2022, 10: 812781.

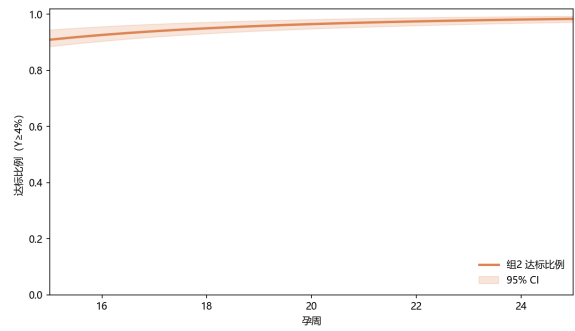
- [2] Deng C, Liu J, Liu S, et al. Maternal and fetal factors influencing fetal fraction[J]. Front Pediatr, 2023, 11: 1066178.
- [3] 吕依蓉, 孙斌, 喻之斌. 基于梯度提升回归树的处理器性能数据挖掘研究 [J]. 集成技术, 2018, 7(5): 47-57.
- [4] 吕佳. 梯度提升回归树算法研究及改进 [D]. 上海: 上海交通大学, 2017.
- [5] 廖慧芬, 邵小兵. 动态规划算法的原理及应用 [J]. 中国科技信息, 2005, (21A): 42-42.
- [6] Ladner R E, Fischer M J. Parallel prefix computation[J]. Journal of the ACM, 1980, 27(4): 831-838.
- [7] World Health Organization. Adolescent pregnancy [EB/OL].[2025-09-06].  
<https://www.who.int/news-room/fact-sheets/detail/adolescent-pregnancy>.
- [8] Miltoft C B, Rode L, Bundgaard J R, Johansen P, Tabor A. Cell-free fetal DNA in the early and late first trimester[J]. Fetal Diagnosis and Therapy, 2020, 47(3): 228-236.
- [9] Lopes J L, Lopes G S, Enninga E A L, et al. Most noninvasive prenatal screens failing due to inadequate fetal cell free DNA are negative for trisomy when repeated[J]. Prenatal Diagnosis, 2020, 40(7): 831-837.
- [10] Ashoor G, Syngelaki A, Poon L, Rezende J C, Nicolaides K H. Fetal fraction in maternal plasma cell-free DNA at 11–13 weeks' gestation: relation to maternal and fetal characteristics[J]. Ultrasound in Obstetrics & Gynecology, 2013, 41(1): 26-32.

**本参赛队未使用任何 AI 工具**

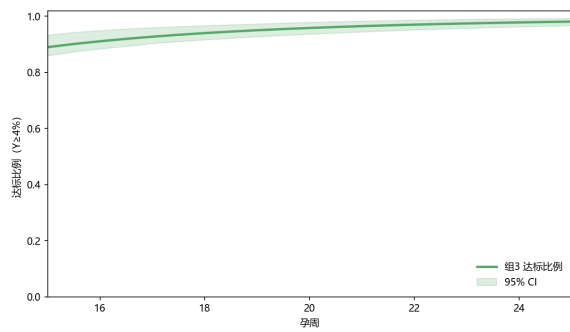
## 附录 A 补充图片



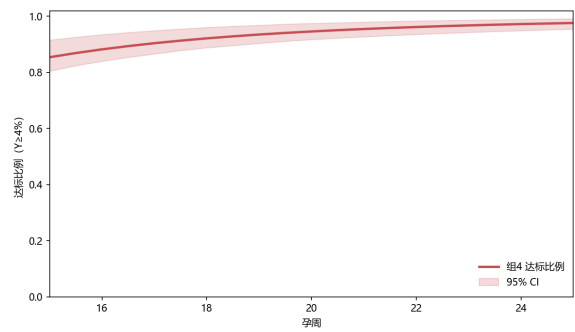
(a) 组 1



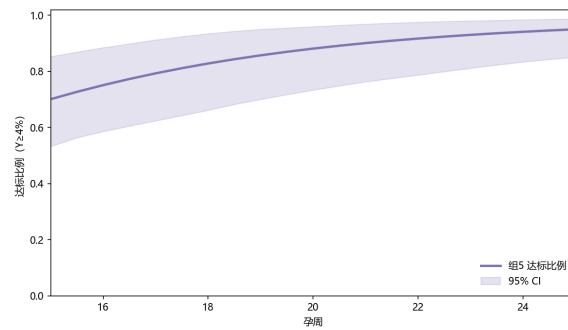
(b) 组 2



(c) 组 3



(d) 组 4



(e) 组 5

图 20 各组单独的达标比例曲线与 95% 置信带



## 附录 B 文件列表

| 文件名       | 功能描述    |
|-----------|---------|
| t1main.py | 问题一主要代码 |
| t2main.py | 问题二主要代码 |
| t3main.py | 问题三主要代码 |
| t4main.py | 问题四主要代码 |

## 附录 C 代码

t1main.py

```
1 def logit(p: np.ndarray) -> np.ndarray:
2     p = np.clip(p, 1e-6, 1 - 1e-6)
3     return np.log(p / (1 - p))
4
5 def inv_logit(z: np.ndarray) -> np.ndarray:
6     return 1.0 / (1.0 + np.exp(-z))
7
8 def build_estimator(n_estimators: int = 1000,
9                     learning_rate: float = 0.03,
10                    max_depth: int = 3,
11                    subsample: float = 0.9,
12                    max_features: float = 0.8,
13                    random_state: int = 88) ->
    TransformedTargetRegressor:
14     prep = Pipeline(steps=[
15         ("qt", QuantileTransformer(output_distribution="normal
16                                     ",
17                                     n_quantiles=200, subsample=
18                                     int(1e9),
19                                     random_state=42)),
20         ("scaler", StandardScaler(with_mean=True, with_std=
21                                   True)),
22         ("poly", PolynomialFeatures(degree=2, include_bias=
23                                   False))
```

```

20     ])
21     gbr = GradientBoostingRegressor(
22         loss="squared_error",
23         n_estimators=int(n_estimators),
24         learning_rate=learning_rate,
25         max_depth=int(max_depth),
26         subsample=subsample,
27         max_features=max_features,
28         random_state=random_state
29     )
30     return TransformedTargetRegressor(
31         regressor=Pipeline([("prep", prep), ("reg", gbr)]),
32         func=logit, inverse_func=inv_logit
33     )
34
35 def kfold_oof_metrics(estimator: TransformedTargetRegressor,
36                       X: np.ndarray, y: np.ndarray,
37                       n_splits: int = 5, seed: int = 42
38                       ) -> Tuple[Dict[str, float], np.ndarray
39                               ]:
40     kf = KFold(n_splits=n_splits, shuffle=True, random_state=
41 seed)
42     oof = np.zeros_like(y, dtype=float)
43     for k, (tr, te) in enumerate(kf.split(X), 1):
44         est_k = clone(estimator)
45         est_k.regressor.named_steps["reg"].random_state = 100
46 + k
47         est_k.fit(X[tr], y[tr])
48         oof[te] = est_k.predict(X[te])
49     rmse = mean_squared_error(y, oof, squared=False)
50     metrics = {
51         "Spearman": float(spearmanr(y, oof).correlation),
52         "MAE": float(mean_absolute_error(y, oof)),
53         "RMSE": float(rmse),
54         "R2": float(r2_score(y, oof))

```

```

52     }
53     return metrics, oof
54
55 def permutation_test_r2(estimator: TransformedTargetRegressor,
56                        X: np.ndarray, y: np.ndarray,
57                        splits: List[Tuple[np.ndarray, np.
58                                     ndarray]]),
59                        n_perm: int = 199, seed: int = 2025
60                        ) -> Tuple[float, float, np.ndarray]:
61     def cv_r2(est, X_, y_, split_):
62         scores = []
63         for tr, te in split_:
64             m = clone(est); m.fit(X_[tr], y_[tr])
65             scores.append(r2_score(y_[te], m.predict(X_[te])))
66         return float(np.mean(scores))
67     observed = cv_r2(estimator, X, y, splits)
68     rng = np.random.RandomState(seed)
69     null = np.empty(n_perm, dtype=float)
70     for b in range(n_perm):
71         null[b] = cv_r2(estimator, X, rng.permutation(y),
72                        splits)
73     p_val = (1.0 + np.sum(null >= observed)) / (n_perm + 1.0)
74     return observed, float(p_val), null
75
76 def permutation_importance_r2(estimator:
77                               TransformedTargetRegressor,
78                               X: np.ndarray, y: np.ndarray,
79                               n_repeats: int = 30, seed: int =
80                               42,
81                               sample_n: int = None
82                               ) -> Tuple[np.ndarray, np.
83                               ndarray]:
84     if sample_n is not None and sample_n < len(y):
85         idx = np.random.RandomState(7).choice(len(y), sample_n
86         , replace=False)

```

```

81         X_, y_ = X[idx], y[idx]
82     else:
83         X_, y_ = X, y
84     pi = permutation_importance(estimator, X_, y_, scoring="r2
85                                ",
86                                n_repeats=n_repeats,
87                                random_state=seed, n_jobs=1)
88     return pi.importances_mean, pi.importances_std
89
90 def partial_dependence_1d(estimator:
91     TransformedTargetRegressor,
92     X: np.ndarray, j: int, grid_points:
93     int = 120
94     ) -> Tuple[np.ndarray, np.ndarray]:
95     xs = np.linspace(X[:, j].min(), X[:, j].max(), grid_points
96     )
97     pdp = []
98     for v in xs:
99         X_mod = X.copy(); X_mod[:, j] = v
100         pdp.append(estimator.predict(X_mod).mean())
101     return xs, np.asarray(pdp)
102
103 def fit_y_fraction_model(df: pd.DataFrame,
104     feature_cols: List[str],
105     target_col: str,
106     n_splits: int = 5) -> Dict[str,
107     object]:
108     work = df[feature_cols + [target_col]].copy()
109     for c in work.columns:
110         work[c] = pd.to_numeric(work[c], errors="coerce")
111     work = work.dropna()
112     y = work[target_col].astype(float).clip(1e-6, 1 - 1e-6).
113     values
114     X = work[feature_cols].astype(float).values
115     est = build_estimator()

```

```

109     oof_metrics, oof_pred = kfold_oof_metrics(est, X, y,
n_splits=n_splits, seed=42)
110     est.fit(X, y)
111     splits = list(KFold(n_splits=n_splits, shuffle=True,
random_state=42).split(X))
112     observed_r2, p_overall, null_r2 = permutation_test_r2(est,
X, y, splits, n_perm=199, seed=2025)
113     imp_mean, imp_std = permutation_importance_r2(est, X, y,
n_repeats=30, seed=42, sample_n=min(2500, len(y)))
114     sp = [(j, abs(spearmanr(X[:, j], y).correlation)) for j in
range(X.shape[1])]
115     top3_idx = [j for j, _ in sorted(sp, key=lambda z: z[1],
reverse=True)[:3]]
116     pdp_dict = {feature_cols[j]: partial_dependence_1d(est, X,
j, grid_points=120) for j in top3_idx}
117     return {
118         "X": X, "y": y,
119         "features": feature_cols,
120         "estimator": est,
121         "oof_metrics": oof_metrics,
122         "oof_pred": oof_pred,
123         "permtest_R2": {"observed": observed_r2, "p_value":
p_overall, "null": null_r2},
124         "perm_importance_R2": {"mean": imp_mean, "std":
imp_std},
125         "top3_pdp": pdp_dict
126     }

```

t2main.py

```

1 def parse_week_str(s):
2     if pd.isna(s): return np.nan
3     s = str(s).strip()
4     m = re.match(r"^s*(\d+)\s*[wW周]?s*\+\s*(\d+)\s*$", s)
5     if m: return int(m.group(1)) + int(m.group(2))/7.0
6     m2 = re.match(r"^s*(\d+)\s*[wW周]\s*$", s)

```

```

7         if m2: return float(m2.group(1))
8         try: return float(s)
9         except: return np.nan
10
11 def safe_sigmoid(x):
12     x = np.clip(x, -50, 50)
13     return 1.0 / (1.0 + np.exp(-x))
14
15 @dataclass
16 class SubjectInterval:
17     pid: str
18     bmi: float
19     L: float
20     R: float
21
22 def load_male_intervals(excel_path: str, sheet_name: str,
23                        thresh: float, h_week: float) -> List[SubjectInterval]:
24     df = pd.read_excel(excel_path, sheet_name=sheet_name)
25     need = ['身高', '体重', '检测孕周', 'Y染色体浓度', '孕妇代码']
26     miss = [c for c in need if c not in df.columns]
27     if miss: raise ValueError(f"缺少列: {miss}")
28     h = pd.to_numeric(df['身高'], errors="coerce")/100.0
29     w = pd.to_numeric(df['体重'], errors="coerce")
30     df['BMI_calc'] = w/(h**2)
31     df['孕周_周'] = df['检测孕周'].apply(parse_week_str)
32     df['Y达标'] = pd.to_numeric(df['Y染色体浓度'], errors="
33     coerce") >= float(thresh)
34     out: List[SubjectInterval] = []
35     for pid, g in df.groupby('孕妇代码'):
36         gi = g.sort_values('孕周_周').dropna(subset=['孕周_周'
37         ])
38         if gi.empty: continue
39         bmi = float(gi.iloc[0]['BMI_calc'])
40         rch = gi[gi['Y达标']]
41         if not rch.empty:

```

```

39         R = float(rch.iloc[0]['孕周_周'])
40         L = 0.0
41         pre = gi[(gi['孕周_周']<R) & (~gi['Y达标'])]
42         if not pre.empty: L = float(pre['孕周_周'].max())
43     else:
44         L = float(gi['孕周_周'].max())
45         R = float(h_week)
46         if np.isfinite(L) and np.isfinite(R) and L<=R:
47             out.append(SubjectInterval(str(pid), bmi, L, R))
48     return out
49
50 class TimeToReadyModel:
51     def __init__(self, intervals: List[SubjectInterval],
52                  h_week: float, l2=5e-4):
53         self.h_week = float(h_week)
54         self.b = np.array([x.bmi for x in intervals], float)
55         self.L = np.array([x.L for x in intervals], float)
56         self.R = np.array([x.R for x in intervals], float)
57         self.l2 = float(l2)
58         self.params: Optional[np.ndarray] = None
59         self.nll_history: List[float] = []
60
61     def _nll(self, params):
62         m0, m1, m2, s0, s1 = params
63         mu = m0 + m1*self.b + m2*(self.b**2)
64         s = np.exp(s0 + s1*self.b)
65         FL = safe_sigmoid((self.L - mu)/s)
66         mask_int = self.R < (self.h_week - 1e-9)
67         ll_int = 0.0
68         if np.any(mask_int):
69             FR = safe_sigmoid((self.R[mask_int] - mu[mask_int
70 ])/s[mask_int])
71             diff = np.maximum(FR - FL[mask_int], 1e-9)
72             ll_int = np.log(diff).sum()
73         mask_rc = ~mask_int

```

```

72         ll_rc = 0.0
73         if np.any(mask_rc):
74             ll_rc = np.log(np.maximum(1.0 - FL[mask_rc], 1e-9)
75 ).sum()
76         penalty = self.l2 * float(np.sum(params*params))
77         return -(ll_int + ll_rc) + penalty
78
79     def fit(self, maxiter=2000):
80         init = np.array([12.0, 0.05, 0.0, math.log(2.0), 0.0],
81 float)
82         bounds = [(-10, 40), (-1, 1), (-0.2, 0.2), (math.log
83 (0.2), math.log(10.0)), (-0.5, 0.5)]
84         self.nll_history = []
85         def _cb(xk): self.nll_history.append(self._nll(xk))
86         res = minimize(lambda p: self._nll(p), init, method="L
87 -BFGS-B",
88                     bounds=bounds, options={"maxiter":
89 maxiter}, callback=_cb)
90         self.params = res.x
91         if len(self.nll_history)==0 or abs(self.nll_history
92 [-1]-res.fun)>1e-10:
93             self.nll_history.append(res.fun)
94         return res
95
96     def F_hat(self, t, b):
97         if self.params is None: raise RuntimeError("not fitted
98 ")
99         t = np.asarray(t, float); b = np.asarray(b, float)
100         m0, m1, m2, s0, s1 = self.params
101         mu = m0 + m1*b + m2*(b**2)
102         s = np.exp(s0 + s1*b)
103         return safe_sigmoid((t - mu)/s)
104
105 def build_w_time(alpha: float, beta: float, h_week: float):
106     def w_scalar(t):

```



```

100         if t<=12.0: return 0.0
101         if t<=27.0: return alpha*(t-12.0)/15.0
102         return alpha + beta*(t-27.0)/(h_week-27.0)
103     def w_vec(t):
104         t = np.asarray(t, float)
105         z = np.zeros_like(t)
106         m1 = (t>12.0) & (t<=27.0)
107         m2 = (t>27.0)
108         z[m1] = alpha*(t[m1]-12.0)/15.0
109         z[m2] = alpha + beta*(t[m2]-27.0)/(h_week-27.0)
110         return z
111     return np.vectorize(w_scalar), w_scalar
112
113 def expected_w_after_t(b: float, t: float, F_hat, fine_grid:
    np.ndarray, W_FINE: np.ndarray, w_time_scalar):
114     mask = fine_grid > t
115     if not np.any(mask): return W_FINE[-1]
116     u = fine_grid[mask]
117     Fu = F_hat(u, b)
118     Fu_prev = np.concatenate([[F_hat(t, b)], F_hat(u[:-1], b)
119 ])
120     dens = np.clip(Fu - Fu_prev, 0.0, 1.0)
121     tail = 1.0 - F_hat(t, b)
122     if tail < 1e-8: return w_time_scalar(t)
123     return float(np.sum(W_FINE[mask] * dens) / tail)
124
125 def risk_at_times_for_b(b: float, t_vec: np.ndarray, F_hat,
    c_retest: float,
126                             fine_grid: np.ndarray, W_FINE: np.
    ndarray, w_time_vec, w_time_scalar):
127     Ft = F_hat(t_vec, b)
128     not_ready = 1.0 - Ft
129     ew = np.array([expected_w_after_t(b, float(t), F_hat,
    fine_grid, W_FINE, w_time_scalar) for t in t_vec])
130     return Ft * w_time_vec(t_vec) + not_ready * (c_retest + ew

```

```

130 )
131 def dp_segment_with_ready_constraint(Rmat_sorted: np.ndarray,
132     F_all_sorted: np.ndarray,
133     t_grid: np.ndarray,
134     k_groups: int, tau_ready: float):
135     n, T = Rmat_sorted.shape
136     INF = np.inf
137     prefix_R = np.vstack([np.zeros(T), np.cumsum(Rmat_sorted,
138         axis=0)])
139     prefix_F = np.vstack([np.zeros(T), np.cumsum(F_all_sorted,
140         axis=0)])
141     CostStar = np.full((n, n), INF); TArg = np.full((n, n),
142         -1)
143     for i in range(n):
144         for j in range(i, n):
145             seg_R = prefix_R[j+1]-prefix_R[i]
146             meanF = (prefix_F[j+1]-prefix_F[i])/(j-i+1)
147             feas = (meanF >= tau_ready)
148             if not np.any(feas): continue
149             seg_R = np.where(feas, seg_R, INF)
150             t_idx = int(np.argmin(seg_R))
151             if not np.isfinite(seg_R[t_idx]): continue
152             CostStar[i,j] = float(seg_R[t_idx]); TArg[i,j] =
153             t_idx
154     dp = np.full((k_groups, n), INF); prev = np.full((k_groups
155         , n), -1)
156     for j in range(n): dp[0,j] = CostStar[0,j]
157     for g in range(1, k_groups):
158         for j in range(g, n):
159             best = INF; bi = -1
160             for i in range(g-1, j):
161                 if not (np.isfinite(dp[g-1,i]) and np.isfinite
162                     (CostStar[i+1,j])): continue
163                 val = dp[g-1,i] + CostStar[i+1,j]

```

```

156         if val < best: best, bi = val, i
157         dp[g,j] = best; prev[g,j] = bi
158     if not np.isfinite(dp[k_groups-1, n-1]): return None, None
159     boundaries, t_idx = [], []
160     g, j = k_groups-1, n-1
161     while g >= 0:
162         i = prev[g,j]; s = 0 if i==-1 else i+1; e = j
163         boundaries.append((s,e)); t_idx.append(TArg[s,e]); j =
164         i; g -= 1
165     boundaries.reverse(); t_idx.reverse()
166     t_vals = [float(t_grid[idx]) for idx in t_idx]
167     for k in range(1,len(t_vals)):
168         if t_vals[k] < t_vals[k-1]:
169             t_vals[k] = t_vals[k-1]
170             t_idx[k] = int(np.argmin(np.abs(t_grid - t_vals[k
171             ])))
172     return boundaries, t_idx
173
174 def dp_segment_unconstrained(Rmat_sorted: np.ndarray, t_grid:
175     np.ndarray, k_groups: int):
176     n, T = Rmat_sorted.shape
177     prefix_R = np.vstack([np.zeros(T), np.cumsum(Rmat_sorted,
178     axis=0)])
179     CostStar = np.full((n, n), np.inf); TArg = np.full((n, n),
180     -1)
181     for i in range(n):
182         for j in range(i, n):
183             seg_R = prefix_R[j+1]-prefix_R[i]
184             idx = int(np.argmin(seg_R))
185             CostStar[i,j] = float(seg_R[idx]); TArg[i,j] = idx
186     dp = np.full((k_groups,n), np.inf); prev = np.full((
187     k_groups,n), -1)
188     for j in range(n): dp[0,j] = CostStar[0,j]
189     for g in range(1,k_groups):
190         for j in range(g,n):

```

```

185         best=np.inf; bi=-1
186         for i in range(g-1,j):
187             val = dp[g-1,i] + CostStar[i+1,j]
188             if val<best: best,bi=val,i
189             dp[g,j]=best; prev[g,j]=bi
190     boundaries, t_idx = [], []
191     g, j = k_groups-1, n-1
192     while g>=0:
193         i=prev[g,j]; s=0 if i==-1 else i+1; e=j
194         boundaries.append((s,e)); t_idx.append(TArg[s,e]); j=i
; g-=1
195     boundaries.reverse(); t_idx.reverse()
196     t_vals = [float(t_grid[idx]) for idx in t_idx]
197     for k in range(1,len(t_vals)):
198         if t_vals[k] < t_vals[k-1]:
199             t_vals[k] = t_vals[k-1]
200             t_idx[k] = int(np.argmin(np.abs(t_grid - t_vals[k
201         ])))
202     return boundaries, t_idx
203
204 def optimize_groups_from_intervals(intervals: List[
205     SubjectInterval],
206                                     h_week: float,
207                                     k: int,
208                                     tau_ready: float,
209                                     alpha: float,
210                                     beta: float,
211                                     c_retest: float,
212                                     tmin: float,
213                                     tmax: float,
214                                     tstep: float,
215                                     fine_step: float
216                                     ) -> Dict[str, object]:
217     model = TimeToReadyModel(intervals, h_week=h_week, l2=5e
218     -4)

```

```

216     model.fit(maxiter=2000)
217     T_GRID = np.round(np.arange(tmin, tmax + tstep/2, tstep),
218 5)
219     FINE_GRID = np.round(np.arange(8.0, h_week, fine_step), 5)
219     w_vec, w_scalar = build_w_time(alpha, beta, h_week)
220     W_FINE = w_vec(FINE_GRID)
221     b = np.array([s.bmi for s in intervals], float)
222     pids = [s.pid for s in intervals]
223     Rmat = np.zeros((len(intervals), len(T_GRID)), float)
224     for i, s in enumerate(intervals):
225         Rmat[i,:] = risk_at_times_for_b(s.bmi, T_GRID, model.
F_hat, c_retest, FINE_GRID, W_FINE, w_vec, w_scalar)
226     order = np.argsort(b)
227     b_sorted = b[order]
228     pid_sorted = [pids[i] for i in order]
229     R_sorted = Rmat[order,:]
230     F_sorted = np.vstack([model.F_hat(T_GRID, bi) for bi in
b_sorted])
231     bnd, tidx = dp_segment_with_ready_constraint(R_sorted,
F_sorted, T_GRID, k, tau_ready)
232     constrained = True
233     if bnd is None:
234         bnd, tidx = dp_segment_unconstrained(R_sorted, T_GRID,
k)
235         constrained = False
236     groups, assign = [], []
237     for g, ((s_idx, e_idx), ti) in enumerate(zip(bnd, tidx),
start=1):
238         bmin = float(b_sorted[s_idx]); bmax = float(b_sorted[
e_idx]); t_rec = float(T_GRID[ti])
239         groups.append({"组别": g, "BMI下界": round(bmin,2), "
BMI上界": round(bmax,2), "建议检测周": t_rec})
240         for ii in range(s_idx, e_idx+1):
241             assign.append({"组别": g, "孕妇代码": pid_sorted[
ii], "BMI": round(float(b_sorted[ii]),2), "建议检测周":

```

```

t_rec}))
242     return {
243         "constrained": constrained,
244         "params": model.params,
245         "groups": pd.DataFrame(groups),
246         "assignments": pd.DataFrame(assign),
247         "T_GRID": T_GRID,
248         "R_sorted": R_sorted,
249         "b_sorted": b_sorted
250     }
251
252 def optimize_groups_from_excel(excel_path: str,
253                                sheet_name: str = "男胎检测数据
254                                ",
255                                thresh: float = 0.04,
256                                h_week: float = 30.0,
257                                k: int = 5,
258                                tau_ready: float = 0.95,
259                                alpha: float = 1.0,
260                                beta: float = 8.0,
261                                c_retest: float = 1.0,
262                                tmin: float = 10.0,
263                                tmax: float = 25.0,
264                                tstep: float = 0.5,
265                                fine_step: float = 0.25
266                                ) -> Dict[str, object]:
267     intervals = load_male_intervals(excel_path, sheet_name,
    thresh, h_week)
    return optimize_groups_from_intervals(intervals, h_week, k
    , tau_ready, alpha, beta, c_retest, tmin, tmax, tstep,
    fine_step)

```

t3main.py

```

1 def normalize_name(s: str) -> str:
2     if s is None: return ""

```

```

3     s = unicodedata.normalize("NFKC", str(s)).lower()
4     return re.sub(r"[\s\-\_\(\)\[\]\{\}\.·\,\,.\.::;; /%]+", "",
5         s)
6
7 def parse_week_str(s):
8     if pd.isna(s): return np.nan
9     if isinstance(s, (int, float)): return float(s)
10    m = re.search(r"(\d+(\.\d+)?)", str(s).strip())
11    return float(m.group(1)) if m else np.nan
12
13 def logit(p: float) -> float:
14     p = float(np.clip(p, 1e-9, 1-1e-9))
15     return math.log(p/(1-p))
16
17 @dataclass
18 class SubjectInterval:
19     pid: str
20     bmi: float
21     age: float
22     z13: float
23     z18: float
24     z21: float
25     L: float
26     R: float
27     right_censored: bool
28
29 def fuzzy_pick(df: pd.DataFrame, prefer, rules):
30     norm = {normalize_name(c): c for c in df.columns}
31     for p in prefer:
32         n = normalize_name(p)
33         if n in norm: return norm[n]
34     for c in df.columns:
35         nc = normalize_name(c)
36         for r in rules:
37             toks = [t for t in r.split("&") if t]

```

```

37         if all(t in nc for t in toks): return c
38     return None
39
40 def pick_columns(df: pd.DataFrame,
41                 col_id=None, col_week=None, col_y=None,
42                 col_bmi=None,
43                 col_height=None, col_weight=None, col_age=
44                 None,
45                 col_z13=None, col_z18=None, col_z21=None):
46     id_col = col_id or fuzzy_pick(df, ["孕妇代码", "编号",
47     "样本号", "pid", "id"], ["孕&妇&代&码", "样&本", "编&号", "id"])
48     week_col = col_week or fuzzy_pick(df, ["孕周", "周数", "检测孕周", "week", "gestational age"], ["孕&周", "检&测&孕&周", "gestational&age", "week"])
49     y_col = col_y or fuzzy_pick(df, ["Y浓度", "Y浓度(%)", "Y%", "Y比例", "Y分数", "y", "chrY", "fetal y"], ["y&浓", "y&%", "chry", "fetal&y", "y&分&数", "y&比&例"])
50     bmi_col = col_bmi or fuzzy_pick(df, ["BMI", "bmi"], ["bmi"])
51     ht_col = col_height or fuzzy_pick(df, ["身高", "身高cm", "height", "height(cm)"], ["身&高", "heigh"])
52     wt_col = col_weight or fuzzy_pick(df, ["体重", "体重kg", "weight", "weight(kg)"], ["体&重", "weigh"])
53     age_col = col_age or fuzzy_pick(df, ["年龄", "age", "年龄(岁)"], ["年&龄", "age"])
54     z13_col = col_z13 or fuzzy_pick(df, ["Z13", "z13"], ["z13"])
55     z18_col = col_z18 or fuzzy_pick(df, ["Z18", "z18"], ["z18"])
56     z21_col = col_z21 or fuzzy_pick(df, ["Z21", "z21"], ["z21"])
57     return {"id": id_col, "week": week_col, "y": y_col, "bmi": bmi_col,
58             "height": ht_col, "weight": wt_col, "age": age_col,
59             "z13": z13_col, "z18": z18_col, "z21": z21_col}

```



```

57
58 def read_base_df(excel: str, sheet: str, overrides: dict) ->
    pd.DataFrame:
59     df = pd.read_excel(excel, sheet_name=sheet)
60     cols = pick_columns(df,
61         col_id=overrides.get("col_id"), col_week=overrides.get(
62             "col_week"),
63         col_y=overrides.get("col_y"), col_bmi=overrides.get("
64             col_bmi"),
65         col_height=overrides.get("col_height"), col_weight=
66             overrides.get("col_weight"),
67         col_age=overrides.get("col_age"), col_z13=overrides.
68             get("col_z13"),
69         col_z18=overrides.get("col_z18"), col_z21=overrides.
70             get("col_z21")
71     )
72     if cols["id"] is None or cols["week"] is None or cols["y"]
73         is None:
74         raise ValueError("缺少 编号/孕周/Y 列")
75     df = df.copy()
76     if cols["bmi"] is not None:
77         df['__BMI__'] = pd.to_numeric(df[cols["bmi"]], errors=
78             'coerce')
79     else:
80         if cols["height"] is None or cols["weight"] is None:
81             raise ValueError("缺少 BMI 或 身高/体重")
82         h_m = pd.to_numeric(df[cols["height"]], errors='coerce
83             ')/100.0
84         w_kg = pd.to_numeric(df[cols["weight"]], errors='
85             coerce')
86         df['__BMI__'] = w_kg / (h_m**2)
87         df['__WEEK__'] = df[cols["week"]].apply(parse_week_str)
88         Yraw = pd.to_numeric(df[cols["y"]], errors='coerce')
89         if np.nanpercentile(Yraw, 90) > 1.5: Yraw = Yraw/100.0
90         df['__Y__'] = Yraw

```

```

82     def num(c): return pd.to_numeric(df[c], errors='coerce')
83     if c is not None else None
84     age = num(cols["age"]); z13 = num(cols["z13"]); z18 = num(
85     cols["z18"]); z21 = num(cols["z21"])
86     df['__AGE__'] = (age.fillna(age.median()) if age is not
87     None else 0.0)
88     df['__Z13__'] = (z13.fillna(np.nanmedian(z13)) if z13 is
89     not None else 0.0)
90     df['__Z18__'] = (z18.fillna(np.nanmedian(z18)) if z18 is
91     not None else 0.0)
92     df['__Z21__'] = (z21.fillna(np.nanmedian(z21)) if z21 is
93     not None else 0.0)
94     df.attrs['pid_col'] = cols["id"]
95     return df
96
97 def intervals_from_df(df: pd.DataFrame, thresh: float, h_week:
98 float) -> List[SubjectInterval]:
99     pid_col = df.attrs.get('pid_col', df.columns[0])
100     out = []
101     for pid, sub in df.groupby(df[pid_col].astype(str)):
102         sub = sub.sort_values('__WEEK__')
103         weeks = sub['__WEEK__'].to_numpy(float)
104         ys = sub['__Y__'].to_numpy(float)
105         bmiv = float(np.nanmedian(sub['__BMI__']))
106         agev = float(np.nanmedian(sub['__AGE__']))
107         z13v = float(np.nanmedian(sub['__Z13__']))
108         z18v = float(np.nanmedian(sub['__Z18__']))
109         z21v = float(np.nanmedian(sub['__Z21__']))
110         if not np.isfinite(bmiv): continue
111         idx = np.where(ys >= thresh)[0]
112         if idx.size>0:
113             j = int(idx[0]); R = float(weeks[j])
114             left = weeks[(weeks < R) & (ys < thresh)]
115             L = float(left.max()) if left.size>0 else 0.0
116             out.append(SubjectInterval(pid, bmiv, agev, z13v,

```

```

110         z18v, z21v, L, R, False))
111         else:
112             L = float(np.nanmax(weeks))
113             out.append(SubjectInterval(pid, bmiv, agev, z13v,
114             z18v, z21v, L, float(h_week), True))
115         return out
116
117 def load_intervals_from_excel(excel: str, sheet: str, thresh:
118 float, h_week: float, overrides: dict) -> List[
119 SubjectInterval]:
120     df = read_base_df(excel, sheet, overrides)
121     return intervals_from_df(df, thresh, h_week)
122
123 class AFTLogLogistic:
124     def __init__(self, l2=0.003, maxiter=1200, verbose=False,
125         pen_align_full=3.0, pen_align_pos=6.0,
126         pen_extra=0.8, pen_sigma=0.006,
127         norm_by_sigma=True):
128         self.l2 = l2; self.maxiter=maxiter; self.verbose=
129         verbose
130         self.pen_align_full=pen_align_full
131         self.pen_align_pos=pen_align_pos
132         self.pen_extra=pen_extra
133         self.pen_sigma=pen_sigma
134         self.norm_by_sigma = norm_by_sigma
135         self.beta_=None; self.s_=None; self.m_q2_=None
136         self.align_mean_pos_ = None; self.align_max_pos_ =
137         None
138
139     @staticmethod
140     def _features(s: SubjectInterval):
141         b = s.bmi
142         return np.array([1.0, b, b*b, s.age, s.z13, s.z18, s.
143         z21], float)

```

```

137     def _fit_q2_baseline(self, b, L, R, right):
138         Xb = np.stack([np.ones_like(b), b, b*b], 1)
139         def nll(theta):
140             m0,m1,m2,s0,s1 = theta
141             mu = Xb @ np.array([m0,m1,m2], float)
142             sig = np.exp(s0 + s1*b)
143             eps=1e-6; Ls=np.maximum(L,eps)
144             zL=(np.log(Ls)-mu)/sig; FL=1/(1+np.exp(-zL))
145             ll=0.0
146             mask=~right
147             if np.any(mask):
148                 Rs=np.maximum(R[mask],eps)
149                 zR=(np.log(Rs)-mu[mask])/sig[mask]; FR=1/(1+np
150 .exp(-zR))
151                 diff=np.maximum(FR-FL[mask],1e-12); ll+=np.sum
152 (np.log(diff))
153             if np.any(right):
154                 surv=np.maximum(1-FL[right],1e-12); ll+=np.sum
155 (np.log(surv))
156                 pen=1e-6*(m0*m0+m1*m1+m2*m2+s0*s0+s1*s1)
157                 return -(ll-pen)
158             init=np.array([0.0,0.0,0.0, math.log(0.6), 0.0])
159             bounds=[(-10,10),(-5,5),(-2,2),(math.log(0.2), math.
160 log(2.5)),(-2,2)]
161             res=minimize(nll, init, method="L-BFGS-B", bounds=
162 bounds, options={"maxiter":1000})
163             return np.array(res.x[:3],float), np.array(res.x[3:],
164 float)
165
166     def _fit_once(self, X, L, R, right, b, Gq2, lam_full,
167 lam_pos, lam_extra, s_q2):
168         n,p = X.shape
169         beta0 = np.zeros(p); beta0[:3] = self.m_q2_
170 scoef0 = s_q2.copy()
171         theta0 = np.concatenate([beta0, scoef0], 0)

```

```

165         idx_extra = np.array([3,4,5,6], int)
166     def nll(theta):
167         beta=theta[:p]; scoef=theta[p:]
168         mu = X @ beta
169         sig = np.exp(scoef[0] + scoef[1]*b)
170         eps=1e-6; Ls=np.maximum(L,eps)
171         zL=(np.log(Ls)-mu)/sig; FL=1/(1+np.exp(-zL))
172         ll=0.0
173         mask=~right
174         if np.any(mask):
175             Rs=np.maximum(R[mask],eps)
176             zR=(np.log(Rs)-mu[mask])/sig[mask]; FR=1/(1+np
177 .exp(-zR))
178             diff=np.maximum(FR-FL[mask],1e-12); ll+=np.sum
179 (np.log(diff))
180         if np.any(right):
181             surv=np.maximum(1-FL[right],1e-12); ll+=np.sum
182 (np.log(surv))
183         delta = (mu - Gq2) / (sig + 1e-6) if self.
184 norm_by_sigma else (mu - Gq2)
185         pen_align = lam_full * float(np.mean(delta**2))
186         pen_align_pos = lam_pos * float(np.mean(np.maximum
187 (delta, 0.0)**2))
188         pen_base = self.l2 * float(np.sum(beta*beta))
189         pen_extra = lam_extra * float(np.sum(beta[
190 idx_extra]*beta[idx_extra]))
191         pen_sigma = self.pen_sigma * float(np.sum(scoef*
192 scoef))
193         return -(ll - pen_base - pen_extra - pen_align -
194 pen_align_pos - pen_sigma)
195         bounds = [(-10,10),(-5,5),(-2,2),(-2,2),(-2,2),(-2,2)
196 ,(-2,2),(math.log(0.2), math.log(2.5)),(-2,2)]
197         res = minimize(nll, theta0, method="L-BFGS-B", bounds=
198 bounds, options={"maxiter": self.maxiter})
199         return res.x

```

```

190
191     def fit(self, intervals, h_week, cont_rounds=6,
cont_growth=2.0, target_mean_pos=0.015, target_max_pos=0.05)
:
192         X = np.vstack([self._features(s) for s in intervals])
193         L = np.array([s.L for s in intervals], float)
194         R = np.array([s.R for s in intervals], float)
195         right = np.array([s.right_censored for s in intervals
], bool)
196         b = X[:,1]
197         m_q2, s_q2 = self._fit_q2_baseline(b, L, R, right)
198         self.m_q2_ = m_q2
199         Xb = np.stack([np.ones_like(b), b, b*b], 1)
200         Gq2 = Xb @ m_q2
201         lam_full = float(self.pen_align_full)
202         lam_pos = float(self.pen_align_pos)
203         lam_extra= float(self.pen_extra)
204         best_theta=None; best_metrics=None
205         for r in range(int(cont_rounds)):
206             theta = self._fit_once(X, L, R, right, b, Gq2,
lam_full, lam_pos, lam_extra, s_q2)
207             beta = theta[:X.shape[1]]; scoef = theta[X.shape
[1]:]
208             mu = X @ beta
209             sig = np.exp(scoef[0] + scoef[1]*b)
210             delta = (mu - Gq2) / (sig + 1e-6) if self.
norm_by_sigma else (mu - Gq2)
211             mean_pos = float(np.mean(np.maximum(delta,0.0)))
212             max_pos = float(np.max(np.maximum(delta,0.0)))
213             best_theta = theta; best_metrics=(mean_pos,
max_pos)
214             if (mean_pos <= target_mean_pos) and (max_pos <=
target_max_pos): break
215             lam_full *= cont_growth; lam_pos *= cont_growth;
lam_extra*= cont_growth**0.5

```

```

216         self.beta_ = best_theta[:X.shape[1]].copy()
217         self.s_      = best_theta[X.shape[1]:].copy()
218         self.align_mean_pos_, self.align_max_pos_ =
best_metrics
219
220     def F(self, t: np.ndarray, subj_or_x) -> np.ndarray:
221         if isinstance(subj_or_x, SubjectInterval): x = self.
_features(subj_or_x)
222         else: x = np.asarray(subj_or_x, float)
223         b = float(x[1])
224         mu = float(np.dot(x, self.beta_))
225         sig = float(np.exp(self.s_[0] + self.s_[1]*b))
226         t = np.asarray(t, float); t = np.maximum(t, 1e-6)
227         z = (np.log(t) - mu) / sig
228         return 1/(1+np.exp(-z))
229
230 def build_w_time(alpha: float, beta: float, h_week: float):
231     def w_scalar(t):
232         if t <= 12.0: return 0.0
233         elif t <= 27.0: return alpha * (t - 12.0) / 15.0
234         else: return alpha + beta * (t - 27.0) / (h_week -
27.0)
235     return np.vectorize(w_scalar), w_scalar
236
237 def expected_w_after_t(Fhat, x, t, fine_grid, W_FINE, w_scalar
):
238     mask = fine_grid > t
239     if not np.any(mask): return 0.0
240     ts = fine_grid[mask]; Ft = Fhat(ts, x)
241     p = np.diff(np.concatenate([[0.0], Ft])); p = np.maximum(p
, 0.0)
242     p = p / (p.sum() + 1e-12)
243     return float(np.sum(p * W_FINE[mask]))
244
245 def risk_for_subject(model, subj, T_GRID, c_retest, FINE_GRID,

```

```

W_FINE, w_vec, w_scalar):
246     x = AFTLogLogistic._features(subj)
247     Ft = model.F(T_GRID, x)
248     not_ready = 1.0 - Ft
249     ew = np.array([expected_w_after_t(model.F, x, float(t),
FINE_GRID, W_FINE, w_scalar) for t in T_GRID])
250     return Ft * w_vec(T_GRID) + not_ready * (c_retest + ew)
251
252 def dp_segment_with_ready_size(R, F, T, K, tau, nmin):
253     n, TT = R.shape; INF=np.inf
254     preR = np.vstack([np.zeros(TT), np.cumsum(R,0)])
255     preF = np.vstack([np.zeros(TT), np.cumsum(F,0)])
256     Cost = np.full((n,n), INF); Arg = np.full((n,n), -1, int)
257     for i in range(n):
258         for j in range(i, n):
259             L = j - i + 1
260             if L < nmin: continue
261             segR = preR[j+1]-preR[i]
262             segF = (preF[j+1]-preF[i]) / L
263             feas = (segF >= tau)
264             if not np.any(feas): continue
265             best=INF; tid=-1
266             for k in np.where(feas)[0]:
267                 c = segR[k]
268                 if c < best: best=c; tid=int(k)
269             Cost[i,j]=float(best); Arg[i,j]=tid
270     dp = np.full((K, n), INF); prev = np.full((K, n), -1, int)
271     for j in range(n):
272         if Arg[0,j]!=-1: dp[0,j]=Cost[0,j]; prev[0,j]=-1
273     for g in range(1, K):
274         for j in range(n):
275             best=INF; bi=-1
276             for i in range(g-1, j):
277                 if dp[g-1,i]==INF or Arg[i+1,j]==-1: continue
278                 v = dp[g-1,i]+Cost[i+1,j]

```



```

279         if v<best: best=v; bi=i
280         dp[g,j]=best; prev[g,j]=bi
281     if not np.isfinite(dp[K-1,n-1]): return None, None, None
282     bounds=[]; tids=[]
283     g=K-1; j=n-1
284     while g>=0:
285         i=prev[g,j]; s=0 if i==-1 else i+1; e=j
286         bounds.append((s,e)); tids.append(Arg[s,e])
287         j=i; g-=1
288     bounds.reverse(); tids.reverse()
289     return bounds, tids, float(dp[K-1,n-1])
290
291 def enforce_monotone_with_gap(ts, bounds, T_GRID, F_sorted,
    tau, gap):
292     out=list(map(float, ts))
293     for k in range(len(bounds)):
294         if k>0 and out[k] < out[k-1]+gap: out[k]=out[k-1]+gap
295         s,e=bounds[k]; tid=int(np.argmin(np.abs(T_GRID-out[k])
    ))
296         if np.mean(F_sorted[s:e+1, tid]) < tau:
297             ok=False
298             for tid2 in range(tid, len(T_GRID)):
299                 if np.mean(F_sorted[s:e+1, tid2]) >= tau:
300                     out[k]=float(T_GRID[tid2]); ok=True; break
301             if not ok: out[k]=float(T_GRID[-1])
302         if k>0 and out[k] < out[k-1]: out[k]=out[k-1]
303     return out
304
305 def optimize_groups_from_intervals(intervals: List[
    SubjectInterval],
306                                   h_week: float,
307                                   k: int,
308                                   tau_ready: float,
309                                   alpha: float,
310                                   beta: float,

```

```

311         c_retest: float,
312         tmin: float,
313         tmax: float,
314         tstep: float,
315         fine_step: float,
316         nmin: int = 1,
317         delta_min: float = 0.0) ->
Dict[str, object]:
318     aft = AFTLogLogistic()
319     aft.fit(intervals, h_week)
320     T_GRID = np.round(np.arange(tmin, tmax + tstep/2, tstep),
321 5)
322     FINE_GRID = np.round(np.arange(8.0, h_week, fine_step), 5)
323     w_vec, w_scalar = build_w_time(alpha, beta, h_week)
324     W_FINE = w_vec(FINE_GRID)
325     order = np.argsort([s.bmi for s in intervals]); ints = [
intervals[i] for i in order]
326     F_sorted = np.vstack([aft.F(T_GRID, AFTLogLogistic.
_features(s)) for s in ints])
327     R_sorted = np.vstack([risk_for_subject(aft, s, T_GRID,
c_retest, FINE_GRID, W_FINE, w_vec, w_scalar) for s in ints
])
328     bounds, tids, total = dp_segment_with_ready_size(R_sorted,
F_sorted, T_GRID, k, tau_ready, nmin)
329     if bounds is None: return {"feasible": False}
330     t_rec = [float(T_GRID[i]) for i in tids]
331     t_adj = enforce_monotone_with_gap(t_rec, bounds, T_GRID,
F_sorted, tau_ready, delta_min)
332     groups, assign = [], []
333     for g, (seg, t0, t1) in enumerate(zip(bounds, t_rec, t_adj
), start=1):
334         s_idx, e_idx = seg
335         bmin = ints[s_idx].bmi; bmax = ints[e_idx].bmi
336         ngrp = e_idx - s_idx + 1
337         ages_grp = np.array([ints[ii].age for ii in range(

```

```

s_idx, e_idx+1)], float)
337     n35 = int(np.sum(ages_grp >= 35.0))
338     p35 = float(n35 / max(ngrp,1))
339     tidx = int(np.argmin(np.abs(T_GRID - t1)))
340     Fbar = float(F_sorted[s_idx:e_idx+1, tidx].mean())
341     groups.append({"组别": g, "BMI下界": round(bmin,2), "
BMI上界": round(bmax,2), "建议检测周": float(t1), "原始最优
周": float(t0), "人数": int(ngrp), "≥35人数": n35, "≥35占比"
: round(p35,3), "组内平均F": round(Fbar,3)})
342     for ii in range(s_idx, e_idx+1):
343         assign.append({"组别":g,"孕妇代码":ints[ii].pid,"
BMI":round(ints[ii].bmi,2),"年龄":float(ints[ii].age),"Z13":
float(ints[ii].z13),"Z18":float(ints[ii].z18),"Z21":float(
ints[ii].z21),"区间L":float(ints[ii].L),"区间R":float(ints[
ii].R),"是否右删":int(ints[ii].right_censored),"建议检测周":
float(t1)})
344     return {"feasible": True, "params": {"beta": aft.beta_, "s
": aft.s_}, "T_GRID": T_GRID, "F_sorted": F_sorted, "
R_sorted": R_sorted, "bounds": bounds, "t_rec": t_rec, "
t_adj": t_adj, "groups": pd.DataFrame(groups), "assignments"
: pd.DataFrame(assign)}
345
346 def optimize_groups_from_excel(excel_path: str,
347     sheet_name: str,
348     thresh: float,
349     h_week: float,
350     k: int,
351     tau_ready: float,
352     alpha: float,
353     beta: float,
354     c_retest: float,
355     tmin: float,
356     tmax: float,
357     tstep: float,
358     fine_step: float,

```

```

359         nmin: int = 1,
360         delta_min: float = 0.0,
361         overrides: Optional[dict] =
None) -> Dict[str, object]:
362     intervals = load_intervals_from_excel(excel_path,
sheet_name, thresh, h_week, overrides or {})
363     return optimize_groups_from_intervals(intervals, h_week, k
, tau_ready, alpha, beta, c_retest, tmin, tmax, tstep,
fine_step, nmin, delta_min)
364
365 def sensitivity_threshold_scan(excel: str, sheet: str,
thresh_list: List[float], h_week: float, k: int, tau_ready:
float, alpha: float, beta: float, c_retest: float, tmin:
float, tmax: float, tstep: float, fine_step: float, nmin:
int, delta_min: float, overrides: Optional[dict] = None) ->
pd.DataFrame:
366     rows=[]
367     for v in thresh_list:
368         ints = load_intervals_from_excel(excel, sheet, v,
h_week, overrides or {})
369         out = optimize_groups_from_intervals(ints, h_week, k,
tau_ready, alpha, beta, c_retest, tmin, tmax, tstep,
fine_step, nmin, delta_min)
370         rows.append({"Y_threshold": v, "feasible": int(out.get
("feasible", False)), "avg_cost": (float(np.nan) if not out.
get("feasible", False) else float(np.nanmean([out["R_sorted"
][s:e+1, int(np.argmin(np.abs(out["T_GRID"]-t)) )].sum() for
(s,e), t in zip(out["bounds"], out["t_adj"])])))/out["
R_sorted"].shape[0]), "t_mean": (float(np.nan) if not out.
get("feasible", False) else float(np.mean(out["t_adj"])))), "
Fbar_mean": (float(np.nan) if not out.get("feasible", False)
else float(np.mean([out["F_sorted"][s:e+1, int(np.argmin(np
.abs(out["T_GRID"]-t)) )].mean() for (s,e), t in zip(out["
bounds"], out["t_adj"])])))))
371     return pd.DataFrame(rows)

```

```

372
373 def load_intervals_with_y_noise(excel: str, sheet: str, thresh
    : float, h_week: float, noise_abs: float, seed: Optional[int
    ], overrides: dict) -> List[SubjectInterval]:
374     df = read_base_df(excel, sheet, overrides)
375     rng = np.random.default_rng(seed)
376     if noise_abs > 0:
377         y = df['__Y__'].to_numpy(dtype=float)
378         y_pert = y + rng.normal(0.0, noise_abs, size=y.shape)
379         df['__Y__'] = np.clip(y_pert, a_min=0.0, a_max=None)
380     return intervals_from_df(df, thresh, h_week)
381
382 def sensitivity_noise_scan(excel: str, sheet: str, thresh:
    float, noise_values: List[float], reps: int, seed: int,
    h_week: float, k: int, tau_ready: float, alpha: float, beta:
    float, c_retest: float, tmin: float, tmax: float, tstep:
    float, fine_step: float, nmin: int, delta_min: float,
    overrides: Optional[dict] = None) -> pd.DataFrame:
383     rows=[]
384     for nv in noise_values:
385         for r in range(int(reps)):
386             ints = load_intervals_with_y_noise(excel, sheet,
    thresh, h_week, nv, int(seed)+r, overrides or {})
387             out = optimize_groups_from_intervals(ints, h_week,
    k, tau_ready, alpha, beta, c_retest, tmin, tmax, tstep,
    fine_step, nmin, delta_min)
388             rows.append({"noise_abs": nv, "rep": r, "feasible"
    : int(out.get("feasible", False)), "avg_cost": (float(np.nan
    ) if not out.get("feasible", False) else float(np.nanmean([
    out["R_sorted"][s:e+1, int(np.argmin(np.abs(out["T_GRID"]-t)
    ) )].sum() for (s,e), t in zip(out["bounds"], out["t_adj"])]
    ))/out["R_sorted"].shape[0]), "t_mean": (float(np.nan) if
    not out.get("feasible", False) else float(np.mean(out["t_adj
    "]))), "Fbar_mean": (float(np.nan) if not out.get("feasible"
    , False) else float(np.mean([out["F_sorted"][s:e+1, int(np.

```

389

```

    argmin(np.abs(out["T_GRID"]-t)) )].mean() for (s,e), t in
    zip(out["bounds"], out["t_adj"])])))))
    return pd.DataFrame(rows)

```

t4main.py

```

1 COLS = {
2     "ID": ["孕妇代码", "样本编号", "ID", "id"],
3     "GA": ["检测孕周", "孕周", "GA"],
4     "BMI": ["孕妇BMI", "BMI"],
5     "reads": ["原始读段数", "总读段数", "reads"],
6     "map_rate": ["在参考基因组上比对的比例"],
7     "dup_rate": ["重复读段的比例"],
8     "filter_rate": ["被过滤掉读段数的比例"],
9     "unique_reads": ["唯一比对的读段数 ", "唯一比对的读段数"],
10    "GC_all": ["GC含量", "GC整体", "GC%"],
11    "Z_13": ["13号染色体的Z值", "Z13", "Z_13"],
12    "Z_18": ["18号染色体的Z值", "Z18", "Z_18"],
13    "Z_21": ["21号染色体的Z值", "Z21", "Z_21"],
14    "Z_X": ["X染色体的Z值", "Z_X"],
15    "X_frac": ["X染色体浓度"],
16    "GC_13": ["13号染色体的GC含量"],
17    "GC_18": ["18号染色体的GC含量"],
18    "GC_21": ["21号染色体的GC含量"],
19    "AB": ["染色体的非整倍体"],
20    "HEALTH": ["胎儿是否健康"],
21 }
22
23 def pick_col(df: pd.DataFrame, cand: List[str]) -> Optional[
    str]:
24     s = set(df.columns)
25     for k in cand:
26         if k in s: return k
27     norm = {c.replace(" ", ""): c for c in df.columns}
28     for k in cand:
29         kk = k.replace(" ", "")

```

```

30         if kk in norm: return norm[kk]
31     return None
32
33 def map_columns(df: pd.DataFrame) -> Dict[str, Optional[str]]:
34     return {k: pick_col(df, v) for k, v in COLS.items()}
35
36 GA_PATTERNS = [
37     re.compile(r"^s*(\d+)\s*[wW周]\s*\+?\s*(\d+)\s*[dD天]?\s*
38     $"),
39     re.compile(r"^s*(\d+)\s*\+\s*(\d+)\s*$"),
40     re.compile(r"^s*(\d+)\s*[wW周]\s*$"),
41     re.compile(r"^s*(\d+)\s*[dD天]\s*$"),
42 ]
43 def parse_ga(x) -> float:
44     if x is None: return np.nan
45     if isinstance(x, (int, float)): return float(x) if x<=40
46     else float(x)/7.0
47     s = str(x).strip().replace("+", "+").replace(": ", ":").
48     replace(". ", ".")
49     for pat in GA_PATTERNS:
50         m = pat.match(s)
51         if m:
52             if pat in (GA_PATTERNS[0], GA_PATTERNS[1]): return
53             float(m.group(1)) + float(m.group(2))/7.0
54             if pat is GA_PATTERNS[2]: return float(m.group(1))
55             if pat is GA_PATTERNS[3]: return float(m.group(1))
56         /7.0
57     try: return float(s)
58     except: return np.nan
59
60 def to_num(x) -> float:
61     if x is None or (isinstance(x, float) and np.isnan(x)):
62     return np.nan
63     if isinstance(x, (int, float)): return float(x)
64     s = str(x).strip().replace(",", "").replace("% ", "%")

```

```

59     if s.endswith("%"):
60         try: return float(s[:-1])/100.0
61         except: return np.nan
62     try: return float(s)
63     except: return np.nan
64
65 def autoscale_fraction_columns(df: pd.DataFrame, cols: List[
    str]):
66     for c in cols:
67         if c in df.columns:
68             x = pd.to_numeric(df[c], errors="coerce")
69             if x.dropna().size == 0: continue
70             q90 = np.quantile(x.dropna(), 0.90)
71             if q90 > 1.5 and q90 <= 100.0:
72                 df[c] = x / 100.0
73
74 def clean_numeric_columns(df: pd.DataFrame, cm: Dict[str, str])
    -> pd.DataFrame:
75     numeric = ["BMI", "reads", "map_rate", "dup_rate", "
    filter_rate", "unique_reads",
76                 "GC_all", "Z_13", "Z_18", "Z_21", "Z_X", "X_frac", "
    GC_13", "GC_18", "GC_21"]
77     if cm.get("GA") and cm["GA"] in df.columns:
78         df[cm["GA"]] = df[cm["GA"]].apply(parse_ga)
79     for k in numeric:
80         col = cm.get(k)
81         if col and col in df.columns:
82             df[col] = df[col].apply(to_num)
83     autoscale_fraction_columns(df, [cm[x] for x in ["map_rate"
    , "dup_rate", "filter_rate", "GC_all", "GC_13", "GC_18", "GC_21", "
    X_frac"] if cm.get(x)])
84     return df
85
86 @dataclass
87 class QCThresholds:

```



```

88     gc_low: float; gc_high: float
89     reads_min: float; map_min: float
90     dup_max: float; filter_max: float
91     uniq_min: Optional[float] = None
92
93 def robust_median_mad(x: np.ndarray) -> Tuple[float, float]:
94     x = x[np.isfinite(x)]
95     if x.size==0: return np.nan, np.nan
96     med = np.median(x)
97     mad = np.median(np.abs(x-med))*1.4826
98     if mad<=1e-12:
99         s = np.std(x); mad = s if s>1e-9 else 1.0
100     return med, mad
101
102 def estimate_qc_thresholds(male_neg: pd.DataFrame, cm: Dict[
103     str, str]) -> QCThresholds:
104     def q(v, a, b):
105         if v is None or v not in male_neg.columns: return (
106             None, None, None)
107         arr = male_neg[v].astype(float); arr = arr[np.isfinite
108             (arr)]
109         if arr.size==0: return (None, None, None)
110         return arr.min(), np.quantile(arr, a), np.quantile(arr,
111             b)
112     _, gc1, gc99 = q(cm["GC_all"], 0.01, 0.99)
113     gc_low = float(gc1-0.002) if gc1 is not None else 0.38
114     gc_high = float(gc99+0.002) if gc99 is not None else 0.42
115     _, r10, _ = q(cm["reads"], 0.10, 0.90); reads_min = float(max
116         (3e6, r10 if r10 is not None else 0))
117     _, m1, _ = q(cm["map_rate"], 0.01, 0.99); map_min = float(m1
118         if m1 is not None else 0.75)
119     _, _, d99 = q(cm["dup_rate"], 0.01, 0.99); dup_max = float(
120         d99 if d99 is not None else 0.06)
121     _, _, f99 = q(cm["filter_rate"], 0.01, 0.99); filter_max =
122         float(f99 if f99 is not None else 0.05)

```

```

115     uniq_min=None
116     if cm["unique_reads"] and cm["reads"] and cm["unique_reads"] in male_neg.columns and cm["reads"] in male_neg.columns:
117         ur = (male_neg[cm["unique_reads"]]/male_neg[cm["reads"]]).replace([np.inf,-np.inf],np.nan).clip(0,1)
118         if ur.dropna().size>0: uniq_min = float(np.quantile(ur.dropna(),0.01))
119     return QCThresholds(gc_low,gc_high,reads_min,map_min,dup_max,filter_max,uniq_min)
120
121 def qc0_status(row: pd.Series, cm: Dict[str,str], th: QCThresholds) -> str:
122     bad=0
123     if cm["GC_all"] and pd.notna(row.get(cm["GC_all"],np.nan)):
124         if not (th.gc_low <= row[cm["GC_all"]] <= th.gc_high):
125             bad+=1
126     if cm["reads"] and pd.notna(row.get(cm["reads"],np.nan)):
127         if row[cm["reads"]] < th.reads_min: bad+=1
128     if cm["map_rate"] and pd.notna(row.get(cm["map_rate"],np.nan)):
129         if row[cm["map_rate"]] < th.map_min: bad+=1
130     if cm["dup_rate"] and pd.notna(row.get(cm["dup_rate"],np.nan)):
131         if row[cm["dup_rate"]] > th.dup_max: bad+=1
132     if cm["filter_rate"] and pd.notna(row.get(cm["filter_rate"],np.nan)):
133         if row[cm["filter_rate"]] > th.filter_max: bad+=1
134     if cm["unique_reads"] and cm["reads"]:
135         denom=row.get(cm["reads"],np.nan); numer=row.get(cm["unique_reads"],np.nan)
136         if np.isfinite(denom) and denom>0 and np.isfinite(numer):
137             ur=float(numer)/float(denom)
138             if th.uniq_min is not None and ur < th.uniq_min:

```

```

138     bad+=1
139     if bad==0: return "valid"
140     if bad<=2: return "suspicious"
141     return "invalid"
142
143 @dataclass
144 class XGateParams:
145     k_sigma: float = 3.0
146
147 def detrend(y: pd.Series, X: Optional[pd.DataFrame]) -> pd.
    Series:
148     if X is None or X.shape[1]==0: return y.astype(float).copy
    ()
149     Xf = X.copy()
150     for c in Xf.columns:
151         if Xf[c].isna().any(): Xf[c] = Xf[c].median()
152     yy = y.values.astype(float)
153     try:
154         m = HuberRegressor(); m.fit(Xf.values, yy)
155         return pd.Series(yy - m.predict(Xf.values), index=y.
    index)
156     except Exception:
157         return y.copy()
158
159 def fit_x_gate(male_neg: pd.DataFrame, cm: Dict[str,str]) ->
    Tuple[float,float]:
160     zx = cm["Z_X"]
161     if not zx or zx not in male_neg.columns: return np.nan, np
    .nan
162     covars=[c for c in [cm["GC_all"],cm["reads"],cm["map_rate"
    ],cm["dup_rate"],cm["filter_rate"],cm["BMI"],cm["GA"]] if c
    and c in male_neg.columns]
163     resid = detrend(male_neg[zx].astype(float), male_neg[
    covars] if covars else None)
164     core = resid.copy()

```

```

164     if core.dropna().size>0:
165         hi=np.quantile(core.dropna(),0.995); core=core[core<=
hi]
166     mu=float(np.nanmean(core)); sigma=float(np.nanstd(core,
ddof=1))
167     if not np.isfinite(sigma) or sigma<1e-9:
168         _,mad=robust_median_mad(core.dropna().values); sigma=
float(mad if np.isfinite(mad) else 1.0)
169     return mu,sigma
170
171 def x_gate_status(row: pd.Series, cm: Dict[str,str], mu: float
, sigma: float, xp: XGateParams) -> str:
172     zx=cm["Z_X"]
173     if not zx or zx not in row or not np.isfinite(mu) or not
np.isfinite(sigma): return "unknown"
174     val=row[zx]
175     if not np.isfinite(val): return "unknown"
176     return "pass" if abs(val-mu) <= xp.k_sigma*sigma else "
fail"
177
178 def add_residual_Z(df: pd.DataFrame, ref_neg: pd.DataFrame, cm
: Dict[str,str]) -> pd.DataFrame:
179     covars=[c for c in [cm["GC_all"],cm["reads"],cm["map_rate"
],cm["dup_rate"],cm["filter_rate"],cm["BMI"],cm["GA"]]] if c
and c in ref_neg.columns]
180     for zk in ["Z_13","Z_18","Z_21"]:
181         zc=cm[zk]
182         if not zc or zc not in df.columns: continue
183         if covars:
184             try:
185                 model=HuberRegressor()
186                 Xref=ref_neg[covars].copy().fillna(ref_neg[
covars].median())
187                 zref=ref_neg[zc].astype(float).values
188                 model.fit(Xref.values, zref)

```

```

189         Xdf=df[covars].copy().fillna(ref_neg[covars].
median())
190         pred=model.predict(Xdf.values)
191         df[f"{zc}_resid"]=df[zc].astype(float).values
- pred
192     except Exception:
193         df[f"{zc}_resid"]=df[zc].astype(float).values
194     else:
195         df[f"{zc}_resid"]=df[zc].astype(float).values
196     return df
197
198 def mahalanobis_tail_prob(X_ref: np.ndarray, X_test: np.
ndarray) -> np.ndarray:
199     try:
200         mcd=MinCovDet().fit(X_ref); d2=mcd.mahalanobis(X_test)
201     except Exception:
202         mu=np.nanmean(X_ref,axis=0,keepdims=True); diff=(
X_test-mu); d2=np.sum(diff*diff,axis=1)
203         k=X_ref.shape[1]; return 1.0-chi2.cdf(d2,df=k)
204
205 def robust_feature_tail(X_ref_df: pd.DataFrame, X_test_df: pd.
DataFrame) -> np.ndarray:
206     ps=[]
207     for c in X_ref_df.columns:
208         r=X_ref_df[c].values.astype(float); t=X_test_df[c].
values.astype(float)
209         r=r[np.isfinite(r)]
210         if r.size==0: ps.append(np.ones_like(t)); continue
211         med,mad=robust_median_mad(r); denom=mad if mad>1e-12
else (np.std(r)+1e-9)
212         z=np.abs((t-med)/denom); p=2.0*(1.0-norm.cdf(z))
213         ps.append(np.clip(p,1e-12,1.0))
214         ps=np.array(ps); return 1.0-np.prod(1.0-ps,axis=0)
215
216 def weak_discriminator_scores(X0: np.ndarray, X1: np.ndarray,

```

```

Xf: np.ndarray):
217     y=np.concatenate([np.zeros(len(X0)), np.ones(len(X1))])
218     X=np.vstack([X0,X1])
219     clf=LogisticRegression(max_iter=1000,class_weight="
balanced",solver="lbfgs")
220     clf.fit(X,y)
221     s0=clf.predict_proba(X0)[: ,1]; s1=clf.predict_proba(X1)
[: ,1]; sf=clf.predict_proba(Xf)[: ,1]
222     auc=roc_auc_score(y,np.concatenate([s0,s1])) if len(np.
unique(y))>1 else np.nan
223     ap =average_precision_score(y,np.concatenate([s0,s1])) if
len(np.unique(y))>1 else np.nan
224     return s0,s1,sf,float(auc if np.isfinite(auc) else np.nan)
,float(ap if np.isfinite(ap) else np.nan)
225
226 def top3_outliers_per_row(X_ref_df: pd.DataFrame, X_test_df:
pd.DataFrame) -> List[str]:
227     meds={}; mads={}
228     for c in X_ref_df.columns:
229         r=X_ref_df[c].values.astype(float); r=r[np.isfinite(r)
]
230         med, mad=robust_median_mad(r)
231         if not np.isfinite(mad) or mad<1e-12: mad=np.std(r)+1e
-9
232         meds[c]=med; mads[c]=mad
233     outs=[]
234     for _,row in X_test_df.iterrows():
235         zs={}
236         for c in X_test_df.columns:
237             v=row[c]
238             if not np.isfinite(v): continue
239             z=(v-meds[c])/(mads[c] if mads[c]>0 else 1.0)
240             zs[c]=z
241         if not zs: outs.append(""); continue
242         top=sorted(zs.items(), key=lambda kv: abs(kv[1]),

```

```

reverse=True)[:3]
243     outs.append("; ".join([f"{name}:{'+' if z>=0 else ''}{
z:.2f}" for name,z in top]))
244     return outs
245
246 def quantile_bins(series: pd.Series, q=[0,0.25,0.5,0.75,1.0],
prefix="Q"):
247     s=pd.to_numeric(series,errors="coerce")
248     if s.dropna().size==0:
249         return pd.Series(["UNK"]*len(series), index=series.
index), None
250     qs=np.quantile(s.dropna(), q)
251     for i in range(1,len(qs)):
252         if qs[i]<=qs[i-1]: qs[i]=qs[i-1]+1e-6
253     bins=pd.cut(s,bins=qs,include_lowest=True,labels=[f"{
prefix}{i}" for i in range(1,len(q))])
254     return bins.astype(str).fillna("UNK"), qs
255
256 def apply_bins_with_edges(series: pd.Series, edges, prefix):
257     s = pd.to_numeric(series, errors="coerce")
258     if edges is None or len(edges)<2 or s.dropna().size==0:
259         return pd.Series(["UNK"]*len(series), index=series.
index)
260     ed = np.array(edges).copy()
261     for i in range(1,len(ed)):
262         if ed[i]<=ed[i-1]: ed[i]=ed[i-1]+1e-6
263     bins = pd.cut(s, bins=ed, include_lowest=True,
264                   labels=[f"{prefix}{i}" for i in range(1,len(
ed))])
265     return bins.astype(str).fillna("UNK")
266
267 def compute_thresholds_by_mode(prob_neg_oof: np.ndarray,
268                               bins_ga_neg: pd.Series,
269                               bins_bmi_neg: pd.Series,
270                               fpr_mode: str, fpr_target:

```

```

float, fpr_margin: float,
270         min_bin_n: int, shrink_lambda:
float):
271     res = {"mode": fpr_mode, "global": None, "ga": {}, "bmi":
        {}, "grid": {}},
272         "counts": {"ga": {}, "bmi": {}, "grid": {}}
273     q_level = 1.0 - min(max(fpr_target + max(fpr_margin, 0.0),
        1e-6), 0.9999)
274     tau_global = float(np.quantile(prob_neg_oof, q_level))
275     res["global"] = tau_global
276     def shrink_tau(arr, fallback):
277         n = int(len(arr))
278         if n == 0:
279             return float(fallback), n
280         tau_raw = float(np.quantile(arr, q_level))
281         w = n / (n + shrink_lambda)
282         tau = w * tau_raw + (1.0 - w) * float(fallback)
283         if n < min_bin_n:
284             tau = max(tau, float(fallback))
285         return float(tau), n
286     if fpr_mode in ["ga", "ga_bmi"]:
287         for g in sorted(pd.unique(bins_ga_neg.astype(str))):
288             mask = (bins_ga_neg.astype(str)==g).values
289             tau, n = shrink_tau(prob_neg_oof[mask], tau_global
        )
290             res["ga"][g] = tau; res["counts"]["ga"][g] = n
291     if fpr_mode in ["bmi", "ga_bmi"]:
292         for b in sorted(pd.unique(bins_bmi_neg.astype(str))):
293             mask = (bins_bmi_neg.astype(str)==b).values
294             tau, n = shrink_tau(prob_neg_oof[mask], tau_global
        )
295             res["bmi"][b] = tau; res["counts"]["bmi"][b] = n
296     if fpr_mode=="ga_bmi":
297         for g in sorted(pd.unique(bins_ga_neg.astype(str))):
298             for b in sorted(pd.unique(bins_bmi_neg.astype(str))

```



```

)):
299         mask = ((bins_ga_neg.astype(str)==g) & (
bins_bmi_neg.astype(str)==b)).values
300         tau_gb, n = shrink_tau(prob_neg_oof[mask], max
(res["ga"].get(g, tau_global),
301
res["bmi"].get(b, tau_global),
302
tau_global))
303         tau_gb = float(max(tau_gb, res["ga"].get(g,
tau_global), res["bmi"].get(b, tau_global), tau_global))
304         res["grid"][(g,b)] = tau_gb; res["counts"]["
grid"][(g,b)] = n
305     return res
306
307 def select_threshold_for_row(thres_dict, fpr_mode, gbin:str,
bbin:str):
308     if fpr_mode=="global":
309         return thres_dict["global"]
310     if fpr_mode=="ga":
311         return thres_dict["ga"].get(gbin, thres_dict["global"
])
312     if fpr_mode=="bmi":
313         return thres_dict["bmi"].get(bbin, thres_dict["global"
])
314     if fpr_mode=="ga_bmi":
315         return thres_dict["grid"].get((gbin,bbin),
316         max(thres_dict["ga"].get(gbin, thres_dict["
global"]),
317
thres_dict["bmi"].get(bbin, thres_dict["
global"]),
318
thres_dict["global"]))
319     return thres_dict["global"]
320
321 def q4_core(excel: str,

```

```

322         male_sheet: str = "男胎检测数据",
323         female_sheet: str = "女胎检测数据",
324         fpr_target: float = 0.05,
325         fpr_margin: float = 0.0,
326         fpr_mode_in: str = "auto",
327         min_bin_n: int = 30,
328         shrink_lambda: float = 100.0):
329     male = pd.read_excel(excel, sheet_name=male_sheet)
330     female = pd.read_excel(excel, sheet_name=female_sheet)
331     cm_m = map_columns(male); cm_f = map_columns(female)
332     for k in COLS.keys():
333         if not cm_m.get(k) and cm_f.get(k): cm_m[k] = cm_f[k]
334     male = clean_numeric_columns(male, cm_m)
335     female = clean_numeric_columns(female, cm_m)
336     if cm_m["AB"] is None or cm_m["AB"] not in male.columns:
337         raise ValueError("缺少 AB 列")
338     male["_ABbin"] = (~male[cm_m["AB"]].fillna("").astype(str)
339 .str.strip().eq("")).astype(int)
340     male_neg0 = male[male["_ABbin"]==0].copy()
341     th = estimate_qc_thresholds(male_neg0, cm_m)
342     male["QC0"] = male.apply(lambda r: qc0_status(r, cm_m,
343 th), axis=1)
344     female["QC0"] = female.apply(lambda r: qc0_status(r, cm_m,
345 th), axis=1)
346     male_neg = male[(male["_ABbin"]==0) & (male["QC0"]!="
347 invalid)].copy()
348     male_pos = male[(male["_ABbin"]==1) & (male["QC0"]!="
349 invalid)].copy()
350     mu_x, sigma_x = fit_x_gate(male_neg, cm_m)
351     xp = XGateParams()
352     female["QCX"] = female.apply(lambda r: x_gate_status(r,
353 cm_m, mu_x, sigma_x, xp), axis=1)
354     male_neg = add_residual_Z(male_neg, male_neg, cm_m)
355     male_pos = add_residual_Z(male_pos, male_neg, cm_m)
356     female = add_residual_Z(female, male_neg, cm_m)

```

```

351     def assemble(df: pd.DataFrame, cm: Dict[str, str]) -> pd.
DataFrame:
352         feats=[]
353         for zk in ["Z_13", "Z_18", "Z_21"]:
354             zc=cm[zk]
355             if zc and f"{zc}_resid" in df.columns: feats.
append(f"{zc}_resid")
356             elif zc: feats.append(zc)
357             for k in ["GC_all", "reads", "map_rate", "dup_rate", "
filter_rate", "BMI", "GA"]:
358                 if cm[k]: feats.append(cm[k])
359                 tmp=df[feats].copy()
360                 if cm["reads"]: tmp["log_reads"]=np.log1p(df[cm["reads
"]]
.astype(float))
361                 for c in tmp.columns: tmp[c]=pd.to_numeric(tmp[c],
errors="coerce")
362                 return tmp
363     X0 = assemble(male_neg, cm_m); X1 = assemble(male_pos,
cm_m); Xf = assemble(female, cm_m)
364     med0 = X0.median(numeric_only=True)
365     X0f = X0.fillna(med0); X1f = X1.fillna(med0); Xff = Xf.
fillna(med0)
366     scaler = RobustScaler().fit(X0f)
367     X0s = scaler.transform(X0f); X1s = scaler.transform(X1f);
Xfs = scaler.transform(Xff)
368     p_maha_0 = mahalanobis_tail_prob(X0s, X0s)
369     p_maha_1 = mahalanobis_tail_prob(X0s, X1s)
370     p_maha_f = mahalanobis_tail_prob(X0s, Xfs)
371     X0df = pd.DataFrame(X0s, columns=X0f.columns)
372     X1df = pd.DataFrame(X1s, columns=X1f.columns)
373     Xfdf = pd.DataFrame(Xfs, columns=Xff.columns)
374     p_feat_0 = robust_feature_tail(X0df, X0df)
375     p_feat_1 = robust_feature_tail(X0df, X1df)
376     p_feat_f = robust_feature_tail(X0df, Xfdf)
377     s_disc_0, s_disc_1, s_disc_f, auc_male, ap_male =

```

```

weak_discriminator_scores(X0s, X1s, Xfs)
378     X_fuse_train = np.vstack([np.column_stack([p_maha_0,
p_feat_0, s_disc_0]),
379                               np.column_stack([p_maha_1,
p_feat_1, s_disc_1])])
380     y_fuse_train = np.concatenate([np.zeros_like(p_maha_0,
dtype=int),
381                                   np.ones_like(p_maha_1,
dtype=int)])
382     fuse_clf = LogisticRegression(max_iter=1000, class_weight="
balanced", solver="lbfgs")
383     fuse_clf.fit(X_fuse_train, y_fuse_train)
384     s_fuse_0 = fuse_clf.predict_proba(np.column_stack([
p_maha_0, p_feat_0, s_disc_0]))[:,1]
385     s_fuse_1 = fuse_clf.predict_proba(np.column_stack([
p_maha_1, p_feat_1, s_disc_1]))[:,1]
386     s_fuse_f = fuse_clf.predict_proba(np.column_stack([
p_maha_f, p_feat_f, s_disc_f]))[:,1]
387     s_all = np.concatenate([s_fuse_0, s_fuse_1])
388     y_all = np.concatenate([np.zeros_like(s_fuse_0, dtype=int)
,
389                             np.ones_like(s_fuse_1, dtype=int)
])
390     skf = StratifiedKFold(n_splits=5, shuffle=True,
random_state=42)
391     oof_prob = np.zeros_like(s_all, dtype=float)
392     for tr_idx, va_idx in skf.split(s_all.reshape(-1,1), y_all
):
393         iso = IsotonicRegression(out_of_bounds="clip")
394         iso.fit(s_all[tr_idx], y_all[tr_idx])
395         oof_prob[va_idx] = iso.predict(s_all[va_idx])
396     iso_full = IsotonicRegression(out_of_bounds="clip").fit(
s_all, y_all)
397     prob_fem = iso_full.predict(s_fuse_f)
398     n0 = len(s_fuse_0)

```

```

399     prob_neg_oof = oof_prob[:n0]
400     bins_ga_neg, ga_edges = quantile_bins(male_neg[cm_m["GA"]
]] if cm_m["GA"] else pd.Series([np.nan]*len(male_neg)),
prefix="GAQ")
401     bins_bmi_neg, bmi_edges = quantile_bins(male_neg[cm_m["BMI
"]] if cm_m["BMI"] else pd.Series([np.nan]*len(male_neg)),
prefix="BMIQ")
402     female_ga_bin = apply_bins_with_edges(female[cm_m["GA"]]
if cm_m["GA"] else pd.Series([np.nan]*len(female)), ga_edges
, "GAQ")
403     female_bmi_bin = apply_bins_with_edges(female[cm_m["BMI"]]
if cm_m["BMI"] else pd.Series([np.nan]*len(female)),
bmi_edges, "BMIQ")
404     if fpr_mode_in == "auto":
405         cnt_grid = pd.crosstab(bins_ga_neg, bins_bmi_neg)
406         min_cell = int(cnt_grid.values.min()) if cnt_grid.size
>0 else 0
407         if min_cell >= 80: fpr_mode = "ga_bmi"
408         else:
409             cnt_ga = bins_ga_neg.value_counts()
410             cnt_bmi = bins_bmi_neg.value_counts()
411             if (cnt_ga.min() if len(cnt_ga)>0 else 0) >= 80:
fpr_mode = "ga"
412             elif (cnt_bmi.min() if len(cnt_bmi)>0 else 0) >=
80: fpr_mode = "bmi"
413             else: fpr_mode = "global"
414         else:
415             fpr_mode = fpr_mode_in
416             thres_dict = compute_thresholds_by_mode(prob_neg_oof,
bins_ga_neg.astype
417 (str), bins_bmi_neg.astype(str),
fpr_mode,
418 fpr_target, fpr_margin,
min_bin_n,
419 shrink_lambda)

```

```

420     def calc_real_fpr(prob, mask, tau):
421         if np.sum(mask)==0: return np.nan
422         return float(np.mean(prob[mask] >= tau))
423     if fpr_mode=="global":
424         fpr_real = calc_real_fpr(prob_neg_oof, np.ones_like(
prob_neg_oof, dtype=bool), thres_dict["global"])
425         fpr_by = pd.DataFrame([{"mode":"global","count":len(
prob_neg_oof),"tau":thres_dict["global"],"fpr":fpr_real}])
426     elif fpr_mode=="ga":
427         rows=[]
428         for g in sorted(pd.unique(bins_ga_neg.astype(str))):
429             mask=(bins_ga_neg.astype(str)==g).values
430             tau=thres_dict["ga"].get(g, thres_dict["global"])
431             rows.append({"GA_bin":g, "count":int(np.sum(mask))
, "tau":tau, "fpr":calc_real_fpr(prob_neg_oof, mask, tau)})
432         fpr_by=pd.DataFrame(rows)
433         fpr_real=float(np.nanmean(fpr_by["fpr"]))
434     elif fpr_mode=="bmi":
435         rows=[]
436         for b in sorted(pd.unique(bins_bmi_neg.astype(str))):
437             mask=(bins_bmi_neg.astype(str)==b).values
438             tau=thres_dict["bmi"].get(b, thres_dict["global"])
439             rows.append({"BMI_bin":b, "count":int(np.sum(mask))
), "tau":tau, "fpr":calc_real_fpr(prob_neg_oof, mask, tau)})
440         fpr_by=pd.DataFrame(rows)
441         fpr_real=float(np.nanmean(fpr_by["fpr"]))
442     else:
443         rows=[]
444         for g in sorted(pd.unique(bins_ga_neg.astype(str))):
445             for b in sorted(pd.unique(bins_bmi_neg.astype(str)
)):
446                 mask=((bins_ga_neg.astype(str)==g) & (
bins_bmi_neg.astype(str)==b)).values
447                 tau=thres_dict["grid"].get((g,b), thres_dict["
global"])

```

```

448         rows.append({"GA_bin":g,"BMI_bin":b,"count":
int(np.sum(mask)),"tau":tau,"fpr":calc_real_fpr(prob_neg_oof
, mask, tau)})
449     fpr_by=pd.DataFrame(rows)
450     fpr_real=float(np.nanmean(fpr_by["fpr"])) if len(
fpr_by)>0 else np.nan
451     qc_invalid = (female["QC0"]=="invalid")
452     x_fail      = (female["QCX"]=="fail")
453     used_tau = []
454     decisions = []
455     for i in range(len(female)):
456         if qc_invalid.iloc[i] or x_fail.iloc[i]:
457             used_tau.append(np.nan); decisions.append("无效-复
检"); continue
458         gbin=str(female_ga_bin.iloc[i]) if female_ga_bin is
not None else "UNK"
459         bbin=str(female_bmi_bin.iloc[i]) if female_bmi_bin is
not None else "UNK"
460         tau = select_threshold_for_row(thres_dict, fpr_mode,
gbin, bbin)
461         used_tau.append(tau)
462         decisions.append("建议复检" if prob_fem[i] >= tau else
"通过")
463     out = female.copy()
464     rename_cols = {
465         cm_m.get("ID"): "ID", cm_m.get("GA"): "GA", cm_m.get("
BMI"): "BMI",
466         cm_m.get("GC_all"): "GC_all", cm_m.get("reads"): "
reads",
467         cm_m.get("map_rate"): "map_rate", cm_m.get("dup_rate")
: "dup_rate",
468         cm_m.get("filter_rate"): "filter_rate",
469         cm_m.get("Z_13"): "Z_13", cm_m.get("Z_18"): "Z_18",
cm_m.get("Z_21"): "Z_21",
470         cm_m.get("Z_X"): "Z_X", cm_m.get("X_frac"): "X_frac",

```

```

471         cm_m.get("AB"): "AB", cm_m.get("HEALTH"): "HEALTH"
472     }
473     rename_cols = {k:v for k,v in rename_cols.items() if k in
out.columns and v is not None}
474     out = out.rename(columns=rename_cols)
475     out["QC0"] = female["QC0"]; out["QCX"] = female["QCX"]
476     out["score_mahalanobis"] = p_maha_f
477     out["score_feature_tail"] = p_feat_f
478     out["score_disc_weak"] = s_disc_f
479     out["score_fuse_raw"] = s_fuse_f
480     out["anomaly_prob"] = prob_fem
481     out["GA_bin"] = female_ga_bin
482     out["BMI_bin"] = female_bmi_bin
483     out["tau_used"] = used_tau
484     out["decision"] = decisions
485     out["top_outliers"] = top3_outliers_per_row(pd.DataFrame(
X0s, columns=X0f.columns), pd.DataFrame(Xfs, columns=Xff.
columns))
486     summary = {
487         "fpr_mode": fpr_mode,
488         "fpr_target": fpr_target,
489         "fpr_margin": fpr_margin,
490         "achieved_fpr_on_male_neg_mean": float(fpr_real) if
fpr_real is not None else None,
491         "N_female_total": int(len(out)),
492         "QC_invalid": int((out["QC0"]=="invalid").sum()),
493         "X_fail": int((out["QCX"]=="fail").sum()),
494         "evaluated": int(((out["QC0"]!="invalid") & (out["QCX"
]!="fail")).sum()),
495         "flagged_suspect": int(((out["decision"]=="建议复检")
& (out["QC0"]!="invalid") & (out["QCX"]!="fail")).sum()),
496         "male_fuse_auc": float(roc_auc_score(y_all, s_all)) if
len(np.unique(y_all))>1 else None,
497         "male_fuse_ap": float(average_precision_score(y_all,
s_all)) if len(np.unique(y_all))>1 else None,

```



```
498         "qc_thresholds": {"GC_all": [th.gc_low, th.gc_high], "
reads_min": th.reads_min, "map_min": th.map_min, "dup_max": th
.dup_max, "filter_max": th.filter_max},
499         "x_gate": {"mu": float(mu_x), "sigma": float(sigma_x),
"k_sigma": xp.k_sigma}
500     }
501     return out, thres_dict, fpr_by, summary
```